RENTEASE: PROPERTY RENTAL AND TENANT MANAGEMENT SYSTEM USING ZOHO CREATOR

A PROJECT REPORT

Submitted by

HARINI SANKAR

(240701168)

in partial fulfilment of the End Semester Examination for the course

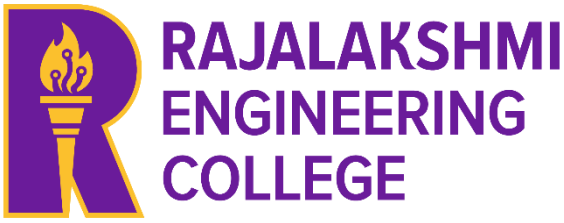
CB23F35 –

LOW-CODE APPLICATION DEVELOPMENT USING ZOHO CREATOR

in

BACHELOR OF ENGINEERING

COMPUTER SCIENCE AND ENGINEERING



RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

Rajalakshmi Nagar, Thandalam, Tamil Nadu – 602 105

NOVEMBER 2026

RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

BONAFIDE CERTIFICATE

Certified that this project report titled "RentEase: Property Rental and Tenant Management System using Zoho Creator" is the Bonafide work of HARINI SANKAR (240701168), who carried out the individual project work on the Zoho Creator low-code platform for the course CB23F35 – Low-Code Application Development using Zoho Creator, during the academic year 2026–2027, Semester V, under my supervision.

SIGNATURE

Dr. Manoranjini J
HEAD OF THE DEPARTMENT
Department of Computer Science & Engineering
Rajalakshmi Engineering College
Thandalam – 602 105

SIGNATURE

Dr. J. Jinu Sophia
COURSE FACULTY
Associate Professor
Department of Computer Science & Engineering
Rajalakshmi Engineering College

Submitted for the End Semester Examination (Individual Project) held on 10th November 2026.

INTERNAL EXAMINER

EXTERNAL EXAMINER

DECLARATION

I, HARINI SANKAR (240701168), hereby declare that the project report titled "RentEase: Property Rental and Tenant Management System using Zoho Creator", submitted for the End Semester Examination of the course CB23F35 – Low-Code Application Development using Zoho Creator, is a record of original work carried out by me individually. The Zoho Creator application described in this report — including its data model, forms, reports, pages, Deluge scripts, workflows and integrations — was designed and developed by me.

I further declare that this work has not been submitted, in part or in full, for the award of any other degree or course credit. Every template, sample application, AI-assisted feature (such as Zia or the AI application builder), code snippet or external resource used in this work has been duly acknowledged in the report.

Place: Chennai
Date: 10th November 2026

SIGNATURE OF THE STUDENT
HARINI SANKAR
240701168

ACKNOWLEDGEMENT

I express my sincere gratitude to our Chairman Thiru. S. Meganathan, Chairperson Dr. Thangam Meganathan and Vice-Chairman Mr. Abhay Shankar Meganathan for providing the infrastructure and facilities that made this work possible.

I am grateful to our Principal, Dr. S. N. Murugesan, Rajalakshmi Engineering College, for the constant encouragement extended to students to take up industry-relevant learning.

I thank Dr. Manoranjini J, Head of the Department, Department of Computer Science and Engineering, for the guidance and support provided throughout the course.

I owe my sincere thanks to my course faculty, Dr. J. Jinu Sophia, Associate Professor, for the valuable guidance, reviews and feedback that shaped this application and report.

I also acknowledge Zoho Corporation for providing access to the Zoho Creator platform and learning resources, and I thank my family and friends for their unwavering support.

HARINI SANKAR

ABSTRACT

Small property owners usually manage rentals with spreadsheets and chat messages. Lease details, rent due dates, repair complaints and deposit refunds are kept in separate places, so overdue rent is noticed late, repairs are forgotten and refunds are disputed. This project addresses that problem for an owner who manages residential and commercial units and needs one reliable system for the manager, the tenant and the owner.

RentEase is a low-code application built on Zoho Creator. It has ten linked forms (Property, Unit, Tenant, Vendor, Lease_Agreement, Rent_Invoice, Rent_Payment, Maintenance_Request, Repair_Approval and Move_Out_Request), nine report types, a dashboard and a command-center page. Eight form workflows written in Deluge fill rent values, generate the lease reference, mark units occupied, mark invoices paid, calculate the deposit refund and send emails. Two reusable functions send receipts and rent reminders. Three schedules run daily, weekly and monthly to flag overdue rent, send a repair digest and generate invoices. A blueprint controls the repair lifecycle with a cost-based branch, two approvals route repair quotes above Rs 5,000 and deposit refunds, and a separate tenant portal limits tenants to the permitted reports.

The application was tested with sample data in the live environment using 18 test cases, of which 16 were executed and passed; two were left for later execution. Defects found during development, such as lookup field naming, missing Auto Number prefix and blueprint status updates, were documented and resolved. The application is used from a single live environment and no external integration was needed. The work shows that a complete rental management process can be built quickly with Zoho Creator, while its limits, such as minimum Number validations and styling of generated pages, call for workarounds.

Keywords: *Low-Code Development; Zoho Creator; Deluge Scripting; Workflow Automation; Blueprint; Approval Process; Customer Portal; Property Management*

PROJECT AT A GLANCE

Attribute	Details
Project Title	RentEase: Property Rental and Tenant Management System
Application Domain	Property rental and tenant management (real estate)
Student Name & Register No.	Harini Sankar – 240701168
Zoho Creator Edition	Zoho Creator free trial (India data center)
Application Name (Link Name)	rentease
Published / Live URL	https://creatorapp.zoho.in/harinisankar.2024.cse_raj17/rentease/
Demo Credentials (per role)	Administrator: account shared with the examiner | Tenant (portal): invited tenant, sign in at https://harinisankar2024cseraj17.zohocreatorportal.in
Number of Forms / Subforms	10 / 2 (Co_Occupants, Deductions)
Number of Reports (by type)	List: 1 Spreadsheet: 1 Kanban: 1 Calendar: 1 Map: 1 Timeline: 1 Pivot: 1 Chart: 1 Summary: 1
Number of Pages / Dashboards	2 (RentEase_Dashboard, RentEase_Command_Center)
Number of Workflows	Form: 8 Schedule: 3 Approval: 2 Blueprint: 1 Batch / Report: 0
Deluge Functions / Custom Actions	2 functions (generateReceipt, sendRentReminders)
Integrations	None (emails use the built-in Creator sendmail task)
Roles & Profiles Defined	Administrator (acts as Manager and Owner), Tenant (portal user, Tenant_Profile)
Environments Used	Single live environment (Stage and Production not used)
Date of Production Deployment	Not applicable (single live environment)

TABLE OF CONTENTS

ABSTRACT iv
PROJECT AT A GLANCE v
LIST OF TABLES ix
LIST OF FIGURES xi
LIST OF ABBREVIATIONS xii
INTRODUCTION 1
1.1 Background 1
1.2 Problem Statement 1
1.3 Objectives 1
1.4 Scope of the Project 2
1.5 Justification for a Low-Code Approach 2
1.6 Organisation of the Report 2
LITERATURE SURVEY AND SYSTEM STUDY 3
2.1 Study of the Existing System 3
2.2 Limitations of the Existing System 3
2.3 Low-Code versus Traditional Development 4
2.4 Literature Survey 4
2.5 Emerging Trends in Low-Code Development 5
2.6 Proposed System 6
REQUIREMENT ANALYSIS 7
3.1 Stakeholders and User Roles 7
3.2 Functional Requirements 7
3.3 Non-Functional Requirements 9
3.4 Hardware and Software Requirements 9
3.5 Use Case Model 10
SYSTEM DESIGN 12
4.1 System Architecture 12
4.2 Data Model 13
4.3 Form Design 14
4.4 Process and Workflow Design 19
4.5 User Interface and Navigation Design 20
4.6 Access Control Design 20
IMPLEMENTATION 22
5.1 Application Setup 22
5.2 Forms and Data Capture 22

5.3 Reports 24
5.4 Pages and Dashboards 26
5.5 Deluge Scripting 27
5.6 Workflows and Automation 29
5.7 Integrations 32
5.8 User Management and Role-Based Access 32
5.9 AI and Analytics Features 33
5.10 Pro-Code Extension 34
5.11 Mobile and Portal Access 34
TESTING AND DEBUGGING 35
6.1 Testing Strategy 35
6.2 Test Cases 35
6.3 Debugging Deluge Scripts 37
6.4 Defect Log 38
6.5 User Acceptance Testing 39
DEPLOYMENT, SECURITY AND GOVERNANCE 40
7.1 Environments and Deployment Strategy 40
7.2 Performance Optimisation 40
7.3 Security Features 41
7.4 Governance and Administration 41
RESULTS AND DISCUSSION 43
8.1 Application Output 43
8.2 Achievement of Objectives 44
8.3 Comparison with the Existing System 45
8.4 Limitations of the Low-Code Approach 45
8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum 46
CONCLUSION AND FUTURE ENHANCEMENTS 47
9.1 Conclusion 47
9.2 Future Enhancements 47
9.3 Learning Outcomes and Career Relevance 47
REFERENCES 48
APPENDIX A – ZOHO CREATOR FEATURE COVERAGE CHECKLIST 49
APPENDIX B – COURSE OUTCOME MAPPING 52
APPENDIX C – COMPLETE DELUGE SOURCE CODE 53
APPENDIX D – USER MANUAL 57
D.1 Accessing the Application 57
D.2 Instructions for the Property Manager 57

D.3 Instructions for the Tenant 57
APPENDIX E – EVALUATION RUBRIC 58
APPENDIX F – VIVA-VOCE RECORD SHEET 60

LIST OF TABLES

Table 2.1: Comparison of Low-Code and Traditional Development for the Proposed Application....4
Table 2.2: Summary of Literature Survey....4
Table 3.1: Stakeholders and User Roles....7
Table 3.2: Functional Requirements....7
Table 3.3: Non-Functional Requirements....9
Table 3.4: Hardware and Software Requirements....9
Table 3.5: Use Case Description – UC-01 Create Lease Agreement....10
Table 3.6: Use Case Description – UC-02 Raise and Approve a Repair....11
Table 4.1: Entities and Relationships....13
Table 4.2: Field Specification – Property Form....14
Table 4.3: Field Specification – Unit Form....15
Table 4.4: Field Specification – Tenant Form....15
Table 4.5: Field Specification – Vendor Form....16
Table 4.6: Field Specification – Lease_Agreement Form....16
Table 4.7: Field Specification – Rent_Invoice Form....17
Table 4.8: Field Specification – Rent_Payment Form....17
Table 4.9: Field Specification – Maintenance_Request Form....17
Table 4.10: Field Specification – Repair_Approval Form....18
Table 4.11: Field Specification – Move_Out_Request Form....18
Table 4.12: Role–Permission Matrix....20
Table 5.1: Reports Implemented....24
Table 5.2: Deluge Scripts Implemented....27
Table 5.3: Workflows Implemented....29
Table 6.1: Test Cases....35
Table 6.2: Defect Log....38
Table 6.3: User Acceptance Feedback....39
Table 7.1: Deployment Checklist....40
Table 7.2: Performance Optimisation Results....41
Table 7.3: Security Controls Implemented....41
Table 8.1: Mapping of Objectives to Outcomes....44
Table 8.2: Existing System versus Proposed Application....45
Table A.1: Feature Coverage Checklist....49
Table B.1: Mapping of Course Outcomes to Report Chapters....52
Table E.1: Evaluation Rubric for the Individual Project....58

Table E.2: Performance Level Descriptors 59
Table F.1: Viva-Voce Questions and Responses 60
Table F.2: Marks Summary 60

LIST OF FIGURES

Figure 2.1: As-Is Process Flow of the Existing Rental Management Process.....3
Figure 3.1: Use Case Diagram of RentEase 10
Figure 4.1: System Architecture of RentEase.....12
Figure 4.2: Entity–Relationship Diagram of RentEase 13
Figure 4.3: Process Flow from Lease Creation to Rent Collection 19
Figure 4.4: Navigation Structure of RentEase 20
Figure 5.1: RentEase Application Home and Left Menu 22
Figure 5.2: Property Form in the Form Builder.....23
Figure 5.3: Property Form in Live Mode.....23
Figure 5.4: Lookup Field Configuration in the Unit Form 23
Figure 5.5: Lease_Agreement Form with Sections, Co Occupants Sub-form and Signature 24
Figure 5.6: Kanban Report: Maintenance_Status_Kanban.....25
Figure 5.7: Calendar Report: Rent_Due_Calendar.....26
Figure 5.8: Pivot Report: Billing_Month_Pivot 26
Figure 5.9: RentEase_Dashboard 27
Figure 5.10: RentEase_Command_Center with Panel, Chart and Gauge 27
Figure 5.11: Form Workflow On_Lease_Created and its Script.....31
Figure 5.12: Blueprint Canvas: Maintenance_Lifecycle_Blueprint.....31
Figure 5.13: Approval Levels and Actions: Repair_Cost_Approval.....31
Figure 5.14: Form Workflow On_Payment_Mark_Paid Calling the Function generateReceipt.....32
Figure 5.15: Tenant_Profile Permission Set for Portal Users.....33
Figure 5.16: Zia Build Agent Used for the Dashboard Page 33
Figure 5.17: Tenant Portal after Sign-in 34
Figure 6.1: Evidence of Workflow Execution: Manager Notification Email.....37
Figure 7.1: Audit Information: Record History and Approvals Inbox 42
Figure 8.1: Scenario Step 1 – Lease created with lease number, reference and QR code 43
Figure 8.2: Scenario Step 2 – Invoices marked Overdue with the late fee.....43
Figure 8.3: Scenario Step 3 – Repair quote waiting in the approval summary 44
Figure 8.4: Scenario Step 4 – Repair_Approval record after approval 44

LIST OF ABBREVIATIONS

Abbreviation	Expansion
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
ER	Entity–Relationship
FR	Functional Requirement
INR	Indian Rupee
KPI	Key Performance Indicator
LCAP	Low-Code Application Platform
NFR	Non-Functional Requirement
QR	Quick Response (code)
RBAC	Role-Based Access Control
REST	Representational State Transfer
SDLC	Software Development Life Cycle
UAT	User Acceptance Testing
UI	User Interface
Zia	Zoho’s AI assistant

CHAPTER 1
INTRODUCTION

1.1 Background

Property rental is a common small business: an owner or a manager lets residential flats, villas and commercial spaces to tenants for a fixed period against a monthly rent and a security deposit. A typical owner handles about 10 to 20 units across several buildings. Each unit needs a lease, a rent invoice every month, a record of payments, repair handling during the tenancy and a deposit settlement when the tenant leaves. RentEase is built for such an owner and for the tenants and vendors involved.

1.2 Problem Statement

Rental records are kept in spreadsheets and chat messages, so the owner has no single reliable view of leases, rent and repairs. The result is missed rent, repairs that are not followed up, and disputes about deposit refunds. The owner needs one system that records the facts once, automates the repeated steps and gives tenants a controlled way to raise requests.

- Rent due dates and late fees are tracked by memory or by manual checking of a spreadsheet.
- Repair complaints arrive by chat and have no stage, owner or cost record.
- Large repair spends are not approved in a consistent way.
- Deposit refunds are calculated on paper and disputed.
- Tenants cannot see their own status without calling the owner.

1.3 Objectives

- To design a data model of ten linked forms for properties, units, tenants, vendors, leases, invoices, payments, maintenance requests, repair approvals and move-out requests.
- To automate rent calculation, lease referencing, unit status, receipts, overdue charges and notifications using Deluge scripting, form workflows and schedules.
- To provide a dashboard and nine report types for the manager, the owner and the tenant.

- To control the repair lifecycle with a blueprint and to route large repair quotes and deposit refunds through approvals.
- To give tenants a separate portal with restricted permissions, and to verify the application with documented test cases.

1.4 Scope of the Project

In scope: ten forms with lookups and two sub-forms; eight form workflows, two functions, three schedules, one blueprint and two approvals; nine reports; a dashboard and a command-center page; a tenant portal with a restricted permission set; testing with sample data. Out of scope: a real payment gateway (payment is recorded in a form and no money moves), SMS or WhatsApp messages, accounting and tax reports, a native mobile application, batch and report workflows, and separate internal user accounts for the manager and the owner (the administrator account stands in for both).

1.5 Justification for a Low-Code Approach

The application is a form-and-workflow system with approvals, reports and a portal, which is exactly what Zoho Creator provides through visual builders. Building it with traditional code would need a database design, a back end, a front end, authentication and hosting. In Zoho Creator the forms, relationships, reports, access control and hosting come with the platform, and only the business rules need Deluge scripts. This shortens development, lets a single developer own the whole application, and keeps the application easy to change when the rules change, for example when the late fee or the approval limit changes.

1.6 Organisation of the Report

Chapter 2 reviews the existing system and related low-code literature. Chapter 3 presents the requirement analysis. Chapter 4 describes the system design. Chapter 5 details the implementation on Zoho Creator. Chapter 6 covers testing and debugging. Chapter 7 discusses deployment, performance optimisation, security and governance. Chapter 8 presents results and discussion, and Chapter 9 concludes the report with future enhancements.

CHAPTER 2
LITERATURE SURVEY AND SYSTEM STUDY

2.1 Study of the Existing System

In the existing process the owner receives complaints and payment news by phone or chat, notes lease details in a spreadsheet, tracks due dates by memory, reminds tenants by phone, follows up repairs over messages and calculates the deposit refund on paper at move-out. Each step depends on one person and on messages that can be lost.

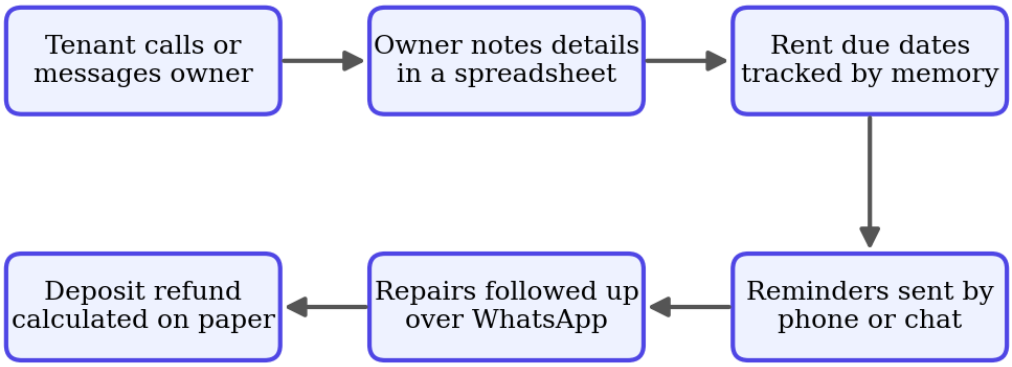


Figure 2.1: As-Is Process Flow of the Existing Rental Management Process

2.2 Limitations of the Existing System

- Rent status is not visible at a glance, so overdue rent is found late and late fees are applied inconsistently.
- Repair requests have no stage, assigned vendor or cost record, and they are easy to forget.
- There is no approval trail for large repair spends or for deposit refunds.
- Data is repeated across spreadsheets and chats, which causes errors and duplicates.
- Tenants have no self-service view and depend on calls and messages.

2.3 Low-Code versus Traditional Development

Table 2.1: Comparison of Low-Code and Traditional Development for the Proposed Application

Criterion	Traditional (Pro-Code) Development	Low-Code on Zoho Creator
Development time	Estimated 8–10 weeks for forms, workflows, approvals, portal and dashboards	Estimated 2–3 weeks using visual builders and Deluge
Skills required	Back-end and front-end programming, SQL, authentication, deployment	Form and report building, basic Deluge scripting
Hosting & infrastructure	Servers, database, backups and deployment pipeline must be arranged	Hosted, backed up and patched by Zoho
Mobile application	Separate mobile or responsive front end to be built	Browser access and the Creator mobile app are available (mobile app not tested here)
Security & access control	Login, roles and permission checks coded by hand	Built-in permission sets and portal login
Maintenance & change	Code change, testing and redeployment for every rule change	Edit forms and workflows in the builder; changes apply to the live application
Customisation limits	Practically unlimited	Bounded by the platform, for example no minimum value on Number fields and limited page styling
Cost	Developer time, servers and licences	Subscription; the free trial was used for this project

2.4 Literature Survey

Table 2.2: Summary of Literature Survey

S.No.	Author(s) / Source & Year	Title / Product	Key Findings	Relevance to this Project
1	W. Rosam and O. Kalnichevski, 2023	Low-Code Development: A Practical Guide (BPB Publications)	Explains low-code concepts, visual modelling and how business users and developers can build applications faster.	Supports the decision to build a business application visually on Zoho Creator.

S.No.	Author(s) / Source & Year	Title / Product	Key Findings	Relevance to this Project
2	Zoho Corporation	Zoho Creator Help Documentation	Describes forms, reports, pages, workflows, blueprints, approvals and portals.	Primary reference for the features used in RentEase.
3	Zoho Corporation	Deluge Script Reference Guide	Defines the syntax of Deluge: fetch, insert, update, loops, functions and sendmail.	Reference for every script written in the project.
4	A. Sahay, A. Indamutsa, D. Di Ruscio and A. Pierantonio, 2020	Supporting the understanding and comparison of low-code development platforms	Compares low-code platforms by features and shows that platforms differ, so choice needs a systematic look.	Justifies choosing Zoho Creator for a form-driven workflow application.
5	Y. Luo, P. Liang, C. Wang, M. Shahin and J. Zhan, 2021	Characteristics and challenges of low-code development: The practitioners' perspective	Reports practitioner views: faster delivery is a benefit, while limited customisation and vendor lock-in are challenges.	Matches the benefits and limits observed in this project (Section 8.4).
6	Gartner, Inc., 2023	Magic Quadrant for Enterprise Low-Code Application Platforms	Evaluates low-code platform vendors and describes the growth of low-code use in enterprises.	Shows that low-code is an established approach for business applications.

2.5 Emerging Trends in Low-Code Development

Low-code platforms are moving towards AI-assisted building. In Zoho Creator, Zia and the Build Agent can create forms, pages and apps from a plain-language description, which was tried in this project for the dashboard page and for a separate application in an earlier experiment. The trend also includes citizen developers who build their own tools, composable applications that combine forms with services, and stronger governance such as roles, environments and audit trails. The experience in this project shows that AI-generated drafts speed up the first version but still need review and manual correction.

2.6 Proposed System

The proposed system is RentEase, a Zoho Creator application with ten linked forms, automatic calculations and notifications, a repair blueprint, two approvals, three schedules, nine reports, a dashboard and a tenant portal. The manager runs daily operations from the application and the dashboard, the tenant uses the portal, and the owner approves large spends from the approvals inbox.

CHAPTER 3
REQUIREMENT ANALYSIS

3.1 Stakeholders and User Roles

Table 3.1: Stakeholders and User Roles

Role	Description	Key Responsibilities in the Application
Property Manager (Administrator)	Runs the rental business day to day.	Maintain properties, units, tenants and vendors; create leases and invoices; record payments; assign and track repairs; send reminders; view dashboards.
Owner	Owns the properties and approves large spends. In the demonstration the administrator account acts as the owner.	Approve or reject repair quotes above Rs 5,000 and deposit refunds from the approvals inbox; review dashboard and reports.
Tenant (Portal user)	External user who rents a unit.	Sign in to the portal; view lease, invoice, payment, maintenance and move-out reports; raise maintenance and move-out requests.

3.2 Functional Requirements

Table 3.2: Functional Requirements

FR ID	Requirement	Role(s)	Priority
FR-01	The system shall allow the manager to register properties, units, tenants and vendors with mandatory fields.	Manager	High
FR-02	The system shall create a lease for a tenant and a unit, fill the monthly rent from the unit, generate a lease number, reference and QR code, capture the tenant signature and mark the unit occupied.	Manager	High
FR-03	The system shall create a monthly rent invoice for every active lease and calculate the total from the rent and the late fee.	Manager	High
FR-04	The system shall flag unpaid invoices past their due date as overdue, add a late fee and email the tenant every day.	System	High
FR-05	The system shall record a payment, mark the invoice paid and email a receipt to the tenant.	Manager	High
FR-06	The system shall allow a tenant to sign in to a portal and raise maintenance and move-out requests.	Tenant	High

FR ID	Requirement	Role(s)	Priority
FR-07	The system shall move a repair request through defined stages, requiring a vendor on assignment and offering the approval stage only when the estimated cost is above Rs 5,000.	Manager	High
FR-08	The system shall route a repair quote above Rs 5,000 to two approval levels and update the approval status on the decision.	Manager, Owner	High
FR-09	The system shall total move-out deductions, calculate the refund and route the refund for approval.	Manager, Owner	Medium
FR-10	The system shall display a dashboard of occupied and vacant units, rent billed and collected, late fees, overdue invoices and open repairs.	Manager, Owner	Medium
FR-11	The system shall email the manager when a new maintenance request is created.	System	Medium
FR-12	The system shall email a weekly digest of open repairs and allow rent reminders to be sent to tenants with unpaid invoices.	Manager	Low

3.3 Non-Functional Requirements

Table 3.3: Non-Functional Requirements

NFR ID	Category	Requirement
NFR-01	Performance	Reports and the dashboard with up to 100 records per form shall load within 3 seconds on a standard broadband connection.
NFR-02	Security	Tenants shall access only the forms and reports allowed by the Tenant_Profile permission set; sign-in shall use the portal login with an emailed invitation.
NFR-03	Usability	The application shall be usable in a standard web browser without training, using dropdown lookups and grouped form sections.
NFR-04	Availability	The application shall run on the Zoho Creator cloud and depend on Zoho's service availability.
NFR-05	Maintainability	Forms, workflows and scripts shall follow a naming convention with underscores, and all scripts shall be documented in Appendix C.
NFR-06	Auditability	Records shall keep the added and modified user, and approvals and blueprint transitions shall keep their history.

3.4 Hardware and Software Requirements

Table 3.4: Hardware and Software Requirements

Category	Specification
Platform	Zoho Creator free trial (India data center, creator.zoho.in)
Scripting language	Deluge (Data Enriched Language for the Universal Grid Environment)
Browser	Google Chrome / Microsoft Edge / Mozilla Firefox (latest version)
API testing tool	Not used (no REST API or webhook is called)
Other Zoho services	None; email is sent with the built-in Creator sendmail task
Mobile	Not tested (access was through the web browser and the tenant portal)
Client hardware	Intel Core i3/i5 or above, 8 GB RAM, internet connection

3.5 Use Case Model

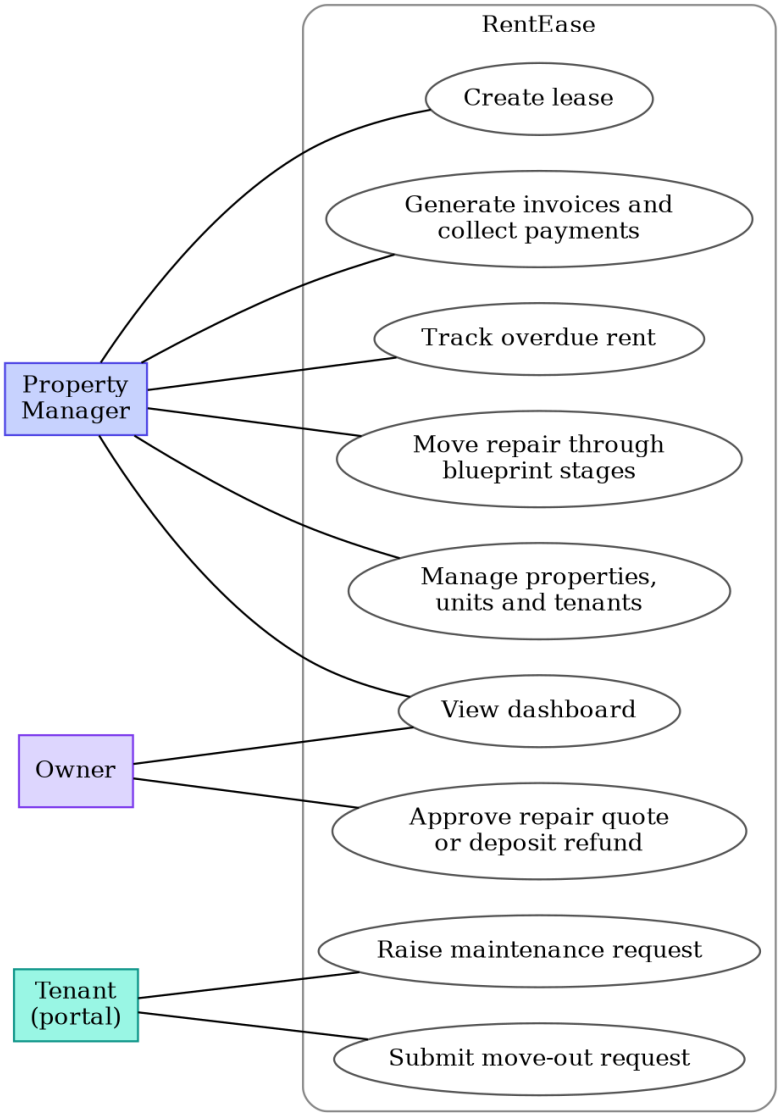


Figure 3.1: Use Case Diagram of RentEase

Table 3.5: Use Case Description – UC-01 Create Lease Agreement

Use Case ID	UC-01
Use Case Name	Create Lease Agreement
Actor(s)	Property Manager
Pre-condition	The property, unit and tenant records exist and the unit is vacant.

Main Flow	1. Manager opens the Lease_Agreement form. 2. Manager selects the tenant and the unit; the monthly rent is filled from the unit. 3. Manager enters the dates, deposit and annual hike, adds co-occupants and the tenant signs. 4. Manager submits the form. 5. The system generates the lease number, reference and QR code, sets the unit to Occupied and emails the tenant.
Alternate Flow	A mandatory field is missing: the system shows an error and does not save the record.
Post-condition	The lease is stored, the unit is Occupied and a welcome email is sent.
Related FR	FR-01, FR-02

Table 3.6: Use Case Description – UC-02 Raise and Approve a Repair

Use Case ID	UC-02
Use Case Name	Raise and Approve a Repair
Actor(s)	Tenant, Property Manager, Owner
Pre-condition	The tenant has an active lease and a portal account.
Main Flow	1. Tenant signs in to the portal and raises a maintenance request with an estimated cost. 2. The system emails the manager. 3. Manager assigns a vendor and starts the work using the blueprint. 4. For a cost above Rs 5,000 the manager sends the request for approval and records the quote. 5. The owner approves the quote at both levels in the approvals inbox. 6. Manager completes the repair and the tenant confirms it.
Alternate Flow	The owner rejects the quote: the approval status becomes Rejected and the requester is informed.
Post-condition	The repair is Closed and every status change is recorded.
Related FR	FR-06, FR-07, FR-08, FR-11

CHAPTER 4
SYSTEM DESIGN

4.1 System Architecture

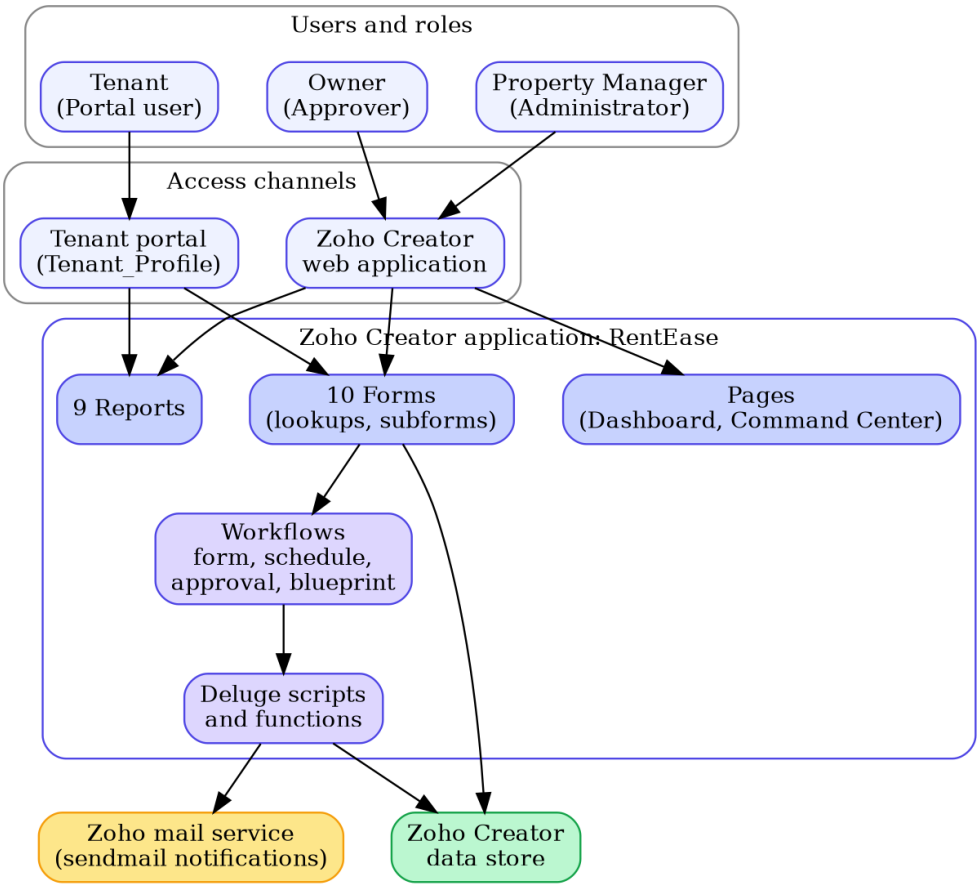


Figure 4.1: System Architecture of RentEase

Users reach RentEase through two channels: the Zoho Creator web application for the manager and owner, and the tenant portal for tenants. Inside the application, forms capture data, reports and pages present it, and workflows and Deluge scripts apply the business rules. All records are stored in the Zoho Creator data store. Notifications are sent through Creator's built-in mail service. The tenant portal is bound to the Tenant_Profile permission set, so tenants reach only the permitted forms and reports.

4.2 Data Model

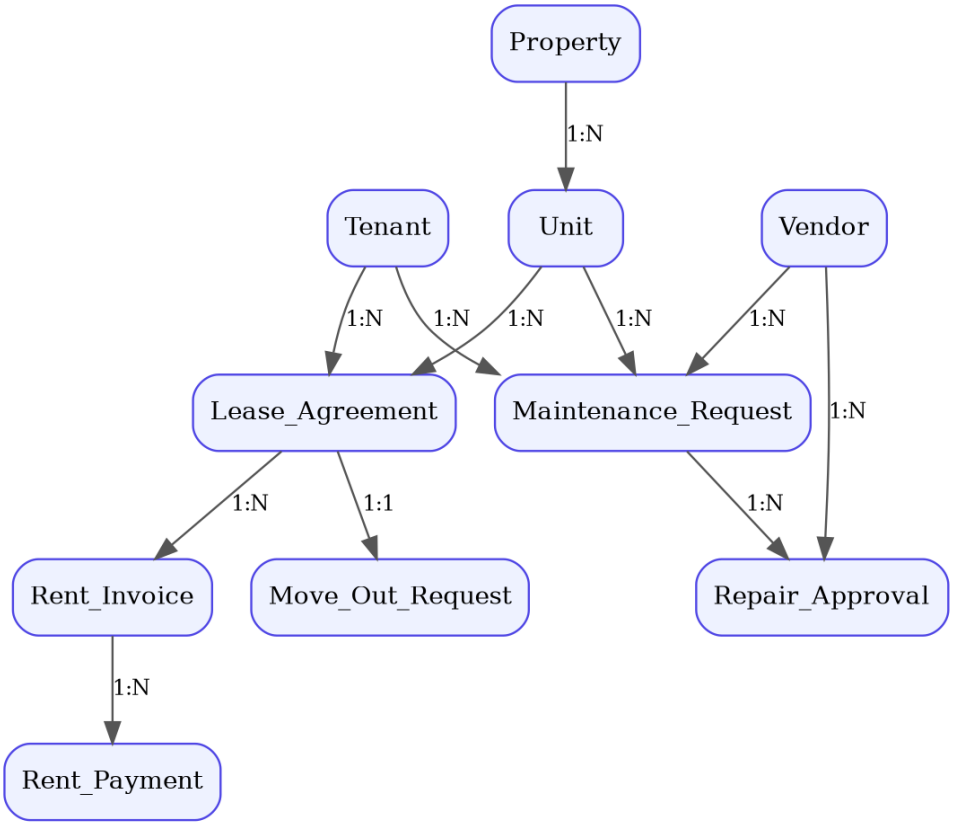


Figure 4.2: Entity–Relationship Diagram of RentEase

Table 4.1: Entities and Relationships

Entity (Form)	Key Fields	Related To	Relationship Type	Implemented Using
Property	Property_Name, Property_Address, Property_Type, Total_Units, Owner_Email	Unit	One-to-Many	Lookup in Unit
Unit	Unit_No, Monthly_Rent, Status, Amenities	Lease_Agreement, Maintenance_Request	One-to-Many	Lookups in Lease_Agreement and Maintenance_Request
Tenant	Tenant_Name, Email, Phone, Occupation	Lease_Agreement, Maintenance_Request	One-to-Many	Lookups in Lease_Agreement and Maintenance_Request
Vendor	Vendor_Name, Service_Type, Rating	Maintenance_Request, Repair_Approval	One-to-Many	Lookups in Maintenance_Request and Repair_Approval

Entity (Form)	Key Fields	Related To	Relationship Type	Implemented Using
Lease_Agreement	Lease_No, Start_Date, End_Date, Monthly_Rent, Security_Deposit	Rent_Invoice, Move_Out_Request	One-to-Many	Lookups in Rent_Invoice and Move_Out_Request; Co_Occupants sub-form
Rent_Invoice	Billing_Month, Due_Date, Rent_Amount, Late_Fee, Total_Amount, Status	Rent_Payment	One-to-Many	Lookup in Rent_Payment
Rent_Payment	Paid_On, Amount_Paid, Mode, Transaction_Ref	Rent_Invoice	Many-to-One	Lookup field Rent_Invoice
Maintenance_Request	Category, Description, Priority, Estimated_Cost, Status	Repair_Approval	One-to-Many	Lookup in Repair_Approval
Repair_Approval	Quoted_Amount, Justification, Approval_Status	Maintenance_Request, Vendor	Many-to-One	Lookups to both forms
Move_Out_Request	Move_Out_Date, Total_Deductions, Refund_Amount, Status	Lease_Agreement	Many-to-One	Lookup field Lease_Agreement; Deductions sub-form

4.3 Form Design

Table 4.2: Field Specification – Property Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Property Name	Property_Name	Single Line	Yes	—	Building name
Property Address	Property_Address	Address	Yes	Map option enabled	Used by the map report
Property Type	Property_Type	Drop Down	No	Apartment, Independent House, Villa, Commercial	
Total Units	Total_Units	Number	Yes	Digits only	No minimum setting in this version

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Property Photo	Property_Photo	Image	No	—	
Owner Name	Owner_Name	Single Line	Yes	—	
Owner Email	Owner_Email	Email	Yes	Email format	Owner contact

Table 4.3: Field Specification – Unit Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Property	Property	Lookup	Yes	Property → Property Name, Drop Down	Parent building
Unit No	Unit_No	Single Line	Yes	—	Unique label such as GM-101
Monthly Rent	Monthly_Rent	Currency	Yes	INR	Source of lease rent
Status	Status	Radio	No	Vacant, Occupied; default Vacant	Set by workflow DS-03
Amenities	Amenities	Multi Select	No	Parking, Lift, Power Backup, Gym, Water Supply, Security	
Furnished	Furnished	Decision Box	No	—	Yes or no

Table 4.4: Field Specification – Tenant Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Tenant Name	Tenant_Name	Name	Yes	First and last name	
Email	Email	Email	Yes	No duplicate values	Portal login ID
Phone	Phone	Phone	Yes	Phone format	
Permanent Address	Permanent_Address	Address	No	—	
Occupation	Occupation	Drop Down	No	Salaried, Self-Employed, Student, Business, Other	
ID Proof	ID_Proof	File Upload	No	—	Identity document
Tenant Photo	Tenant_Photo	Image	No	—	

Table 4.5: Field Specification – Vendor Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Vendor Name	Vendor_Name	Single Line	Yes	—	
Phone	Phone	Phone	Yes	Phone format	
Service Type	Service_Type	Multi Select	No	Plumbing, Electrical, Carpentry, Painting, Cleaning	
Rating	Rating	Decimal	No	Expected range 0 to 5	Range not enforced
Documents	Documents	File Upload	No	—	

Table 4.6: Field Specification – Lease_Agreement Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Lease No	Lease_No	Auto Number	—	System generated	No prefix option in this version
Lease Reference	Lease_Reference	Single Line	No	Admin Only	Set by workflow DS-02 as LEASE-2026-n
Tenant	Tenant	Lookup	Yes	Tenant → Tenant Name	
Unit	Unit	Lookup	Yes	Unit → Unit No	Triggers DS-01
Start Date	Start_Date	Date	Yes	—	
End Date	End_Date	Date	Yes	—	
Monthly Rent	Monthly_Rent	Currency	Yes	INR	Filled by DS-01
Security Deposit	Security_Deposit	Currency	Yes	INR	
Annual Hike	Annual_Hike	Percent	No	—	
Co Occupants	Co_Occupants	SubForm	No	Name, Relation, Age	Repeating rows
Tenant Signature	Tenant_Signature	Signature	Yes	—	
Lease QR	Lease_QR	QR/Barcode	—	Input data ${Lease_No}	Created on save
Agreement PDF	Agreement_PDF	File Upload	No	—	

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Status	Status	Drop Down	No	Draft, Active, Expiring, Closed; default Draft	

Table 4.7: Field Specification – Rent_Invoice Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Lease	Lease_Agreement	Lookup	Yes	Lease Reference	Triggers DS-04
Billing Month	Billing_Month	Single Line	Yes	—	For example October 2026
Due Date	Due_Date	Date	Yes	—	
Rent Amount	Rent_Amount	Currency	Yes	INR	Filled by DS-04
Late Fee	Late_Fee	Currency	No	Default 0	Set to 500 by DS-11
Total Amount	Total_Amount	Formula	—	Rent_Amount + Late_Fee	Calculated by the system
Status	Status	Radio	No	Unpaid, Paid, Overdue; default Unpaid	

Table 4.8: Field Specification – Rent_Payment Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Invoice	Rent_Invoice	Lookup	Yes	Billing Month	Triggers DS-05
Paid On	Paid_On	Date-Time	Yes	Current date and time	
Amount Paid	Amount_Paid	Currency	Yes	INR	Filled by DS-05
Mode	Mode	Radio	No	UPI, Card, Bank Transfer, Cash	
Transaction Ref	Transaction_Ref	Single Line	No	—	
Receipt	Receipt	File Upload	No	—	

Table 4.9: Field Specification – Maintenance_Request Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Tenant	Tenant	Lookup	Yes	Tenant → Tenant Name	
Unit	Unit	Lookup	Yes	Unit → Unit No	

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Category	Category	Drop Down	Yes	Plumbing, Electrical, Carpentry, Painting, Appliance, Other	
Description	Description	Multi Line	Yes	—	
Issue Photo	Issue_Photo	Image	No	—	
Issue Video	Issue_Video	Video	No	—	
Priority	Priority	Radio	No	Low, Medium, High; default Medium	
Estimated Cost	Estimated_Cost	Currency	No	INR	Drives the blueprint branch
Assigned Vendor	Assigned_Vendor	Lookup	No	Vendor → Vendor Name	Mandatory on the Assign transition
Status	Status	Drop Down	No	Raised to Closed; default Raised	Set by blueprint scripts

Table 4.10: Field Specification – Repair_Approval Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Maintenance Request	Maintenance_Request	Lookup	Yes	Category	
Vendor	Vendor	Lookup	Yes	Vendor → Vendor Name	
Quoted Amount	Quoted_Amount	Currency	Yes	INR	Approval criteria: above 5000
Justification	Justification	Multi Line	Yes	—	
Quote File	Quote_File	File Upload	No	—	
Approval Status	Approval_Status	Drop Down	No	Pending, Approved, Rejected; default Pending	Set by the approval

Table 4.11: Field Specification – Move_Out_Request Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Lease	Lease_Agreement	Lookup	Yes	Lease Reference	

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Move Out Date	Move_Out_Date	Date	Yes	—	
Reason	Reason	Multi Line	No	—	
Inspection Photos	Inspection_Photos	Image	No	—	
Deductions	Deductions	SubForm	No	Item, Amount	Repeating rows
Total Deductions	Total_Deductions	Currency	No	INR	Calculated by DS-07
Refund Amount	Refund_Amount	Currency	No	INR	Deposit minus deductions, DS-07
Status	Status	Drop Down	No	Submitted, Inspected, Approved, Refunded; default Submitted	

4.4 Process and Workflow Design

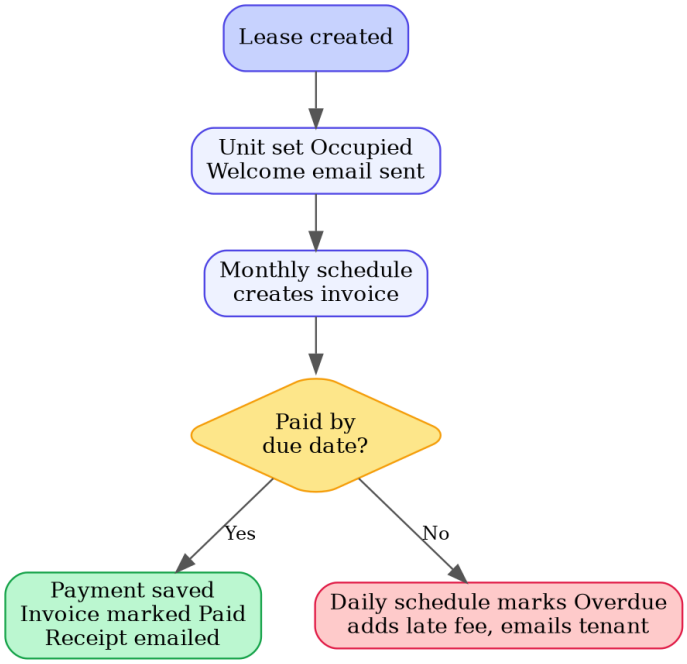


Figure 4.3: Process Flow from Lease Creation to Rent Collection

A new lease sets the unit to Occupied and sends a welcome email. Every month a schedule creates an invoice. If the invoice is not paid by its due date, the daily schedule marks it Overdue, adds the late fee and emails the tenant. A saved payment marks the

invoice Paid and sends a receipt. Repairs and move-out follow their own blueprint and approvals, described in Chapter 5.

4.5 User Interface and Navigation Design

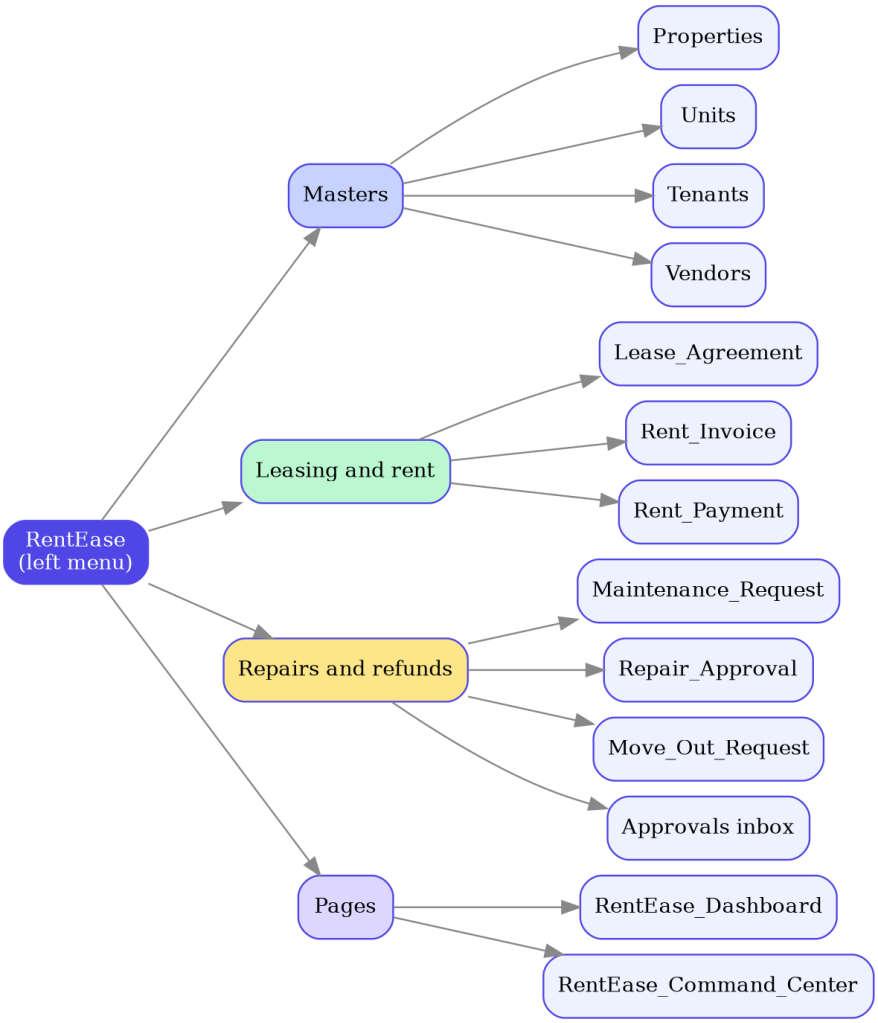


Figure 4.4: Navigation Structure of RentEase

4.6 Access Control Design

Table 4.12: Role–Permission Matrix

Component	Administrator	Manager	Owner	Tenant (Portal)
Form: Property	A E V D	A E V	V	—
Form: Unit	A E V D	A E V	V	—
Form: Tenant	A E V D	A E V	V	—
Form: Vendor	A E V D	A E V	V	—
Form: Lease_Agreement	A E V D	A E V	V	V
Form: Rent_Invoice	A E V D	A E V	V	V

Component	Administrator	Manager	Owner	Tenant (Portal)
Form: Rent_Payment	A E V D	A E V	V	V
Form: Maintenance_Request	A E V D	A E V	V	A V
Form: Repair_Approval	A E V D	A E V	V	—
Form: Move_Out_Request	A E V D	A E V	V	A V
Report: Rent_Invoice Report	V	V	V	V
Report: Rent_Invoices_Spreadsheet	V E	V E	—	—
Page: RentEase_Dashboard	V	V	V	—
Page: RentEase_Command_Center	V	V	—	—
Approvals inbox	Approve / Reject	—	Approve / Reject	—

A = Add, E = Edit, V = View, D = Delete; — = no access. The matrix is the design target. In the implementation the tenant column is enforced through the Tenant_Profile permission set, and the administrator account stands in for the manager and the owner.

CHAPTER 5
IMPLEMENTATION

5.1 Application Setup

A new application named RentEase was created in Zoho Creator from scratch, with the time zone Asia/Kolkata and the date format dd-MMM-yyyy. The Environments option was left off because the project is developed and used in a single live environment. The left menu lists the ten forms with their reports, the approvals inbox and the two pages.

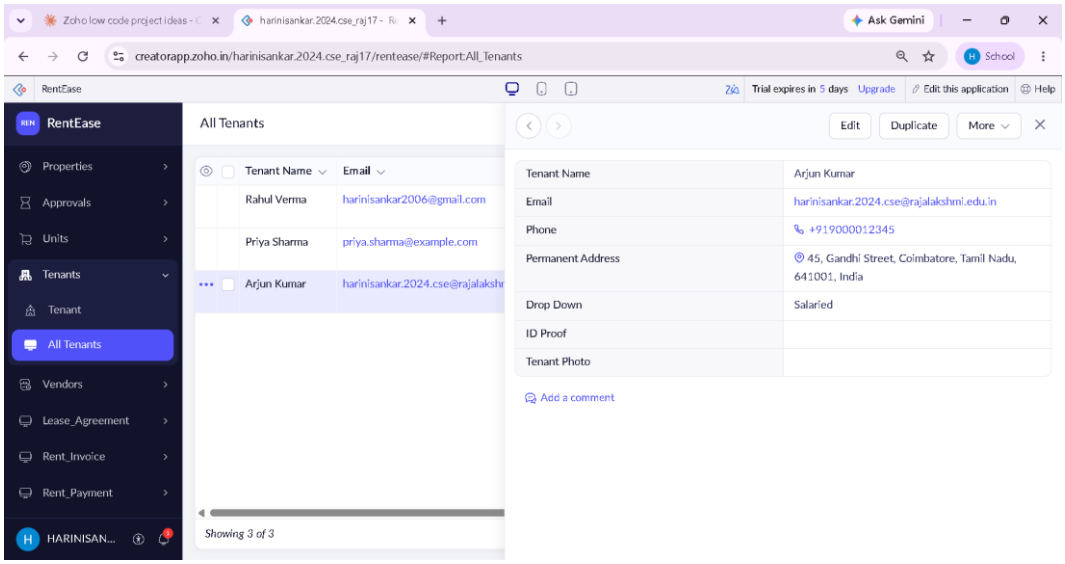


Figure 5.1: RentEase Application Home and Left Menu

5.2 Forms and Data Capture

Ten forms were built in dependency order so that every lookup had a parent form: Property, Unit, Tenant, Vendor, Lease_Agreement, Rent_Invoice, Rent_Payment, Maintenance_Request, Repair_Approval and Move_Out_Request. Lookup fields link the forms, Co_Occupants and Deductions are sub-forms, and the Lease_Agreement form is grouped into sections. Lease_No is an Auto Number, Total_Amount is a Formula field, and Lease_QR is generated from the lease number. Mandatory fields, initial values and the no-duplicate rule on the tenant email protect data quality.

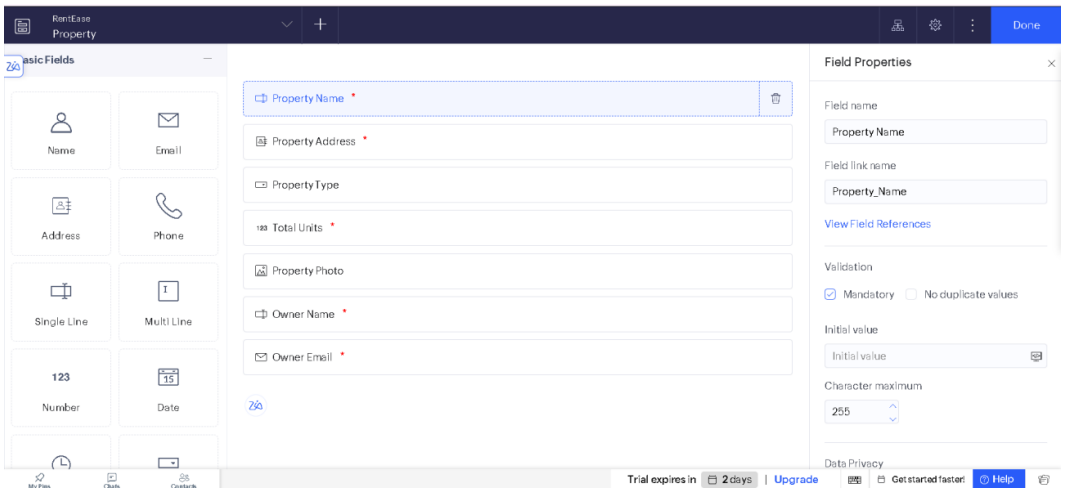


Figure 5.2: Property Form in the Form Builder

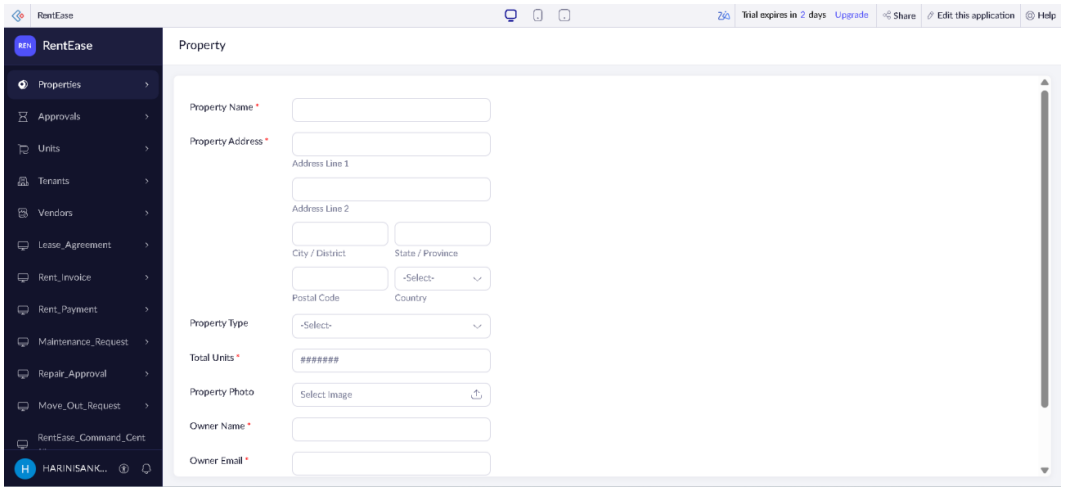


Figure 5.3: Property Form in Live Mode

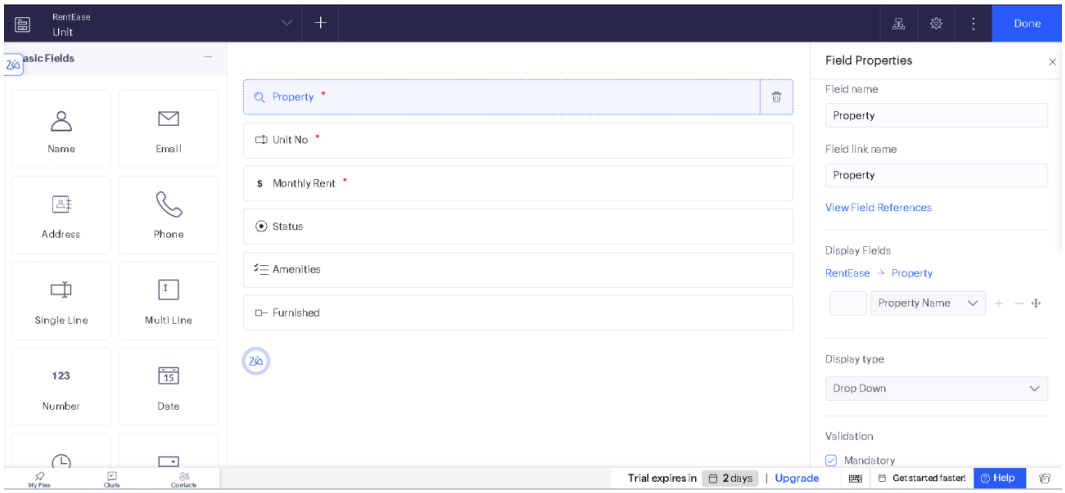


Figure 5.4: Lookup Field Configuration in the Unit Form

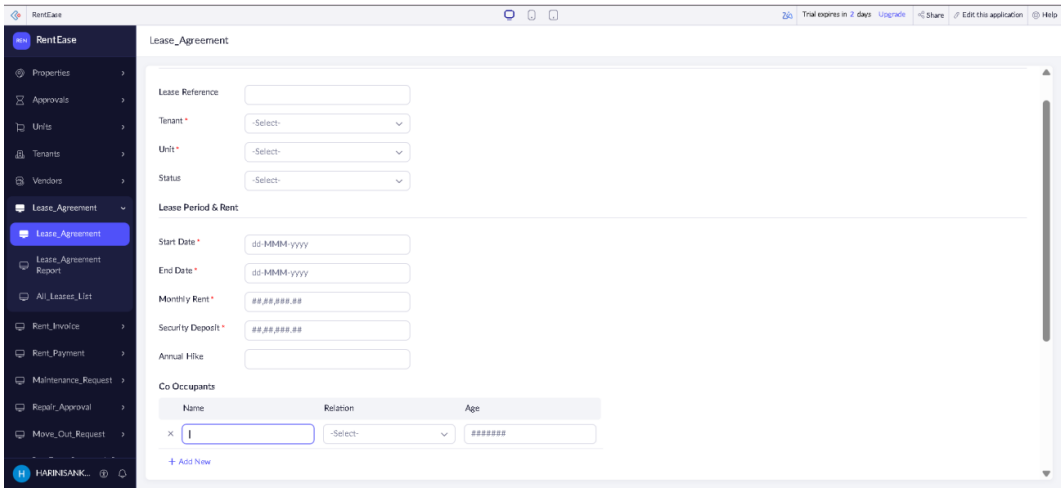


Figure 5.5: Lease_Agreement Form with Sections, Co Occupants Sub-form and Signature

5.3 Reports

Nine reports were built, one for each report type of Zoho Creator, so that each user group sees the data in the format suited to a decision. Table 5.1 lists them.

Table 5.1: Reports Implemented

Report Name	Type	Source Form	Filter / Grouping / Sorting	Custom Actions	Roles
All_Leases_List	List	Lease_Agreement	Columns: Lease No, Lease Reference, Tenant, Unit, Status, dates; search	None	Manager
Rent_Invoices_Spreadsheet	Spreadsheet	Rent_Invoice	Grid of invoices with editable Status	Inline edit	Manager
Maintenance_Status_Kanban	Kanban	Maintenance_Request	Cards grouped by Status	Drag a card to change the stage	Manager
Rent_Due_Calendar	Calendar	Rent_Invoice	Date field: Due Date	None	Manager
Property_Location_Map	Map	Property	Pins from Property Address	None	Manager, Owner

Report Name	Type	Source Form	Filter / Grouping / Sorting	Custom Actions	Roles
Rent_Payment_Timeline	Timeline	Rent_Payment	Date field: Paid On	None	Manager
Billing_Month_Pivot	Pivot	Rent_Invoice	Rows: Billing Month; columns: Status; sum of Total Amount	None	Owner, Manager
Revenue_By_Property_Chart	Chart	Unit	Bar chart of Monthly Rent for each Property	None	Owner
Property_Wise_Summary	Summary	Unit	Grouped by Property; total Monthly Rent	None	Owner

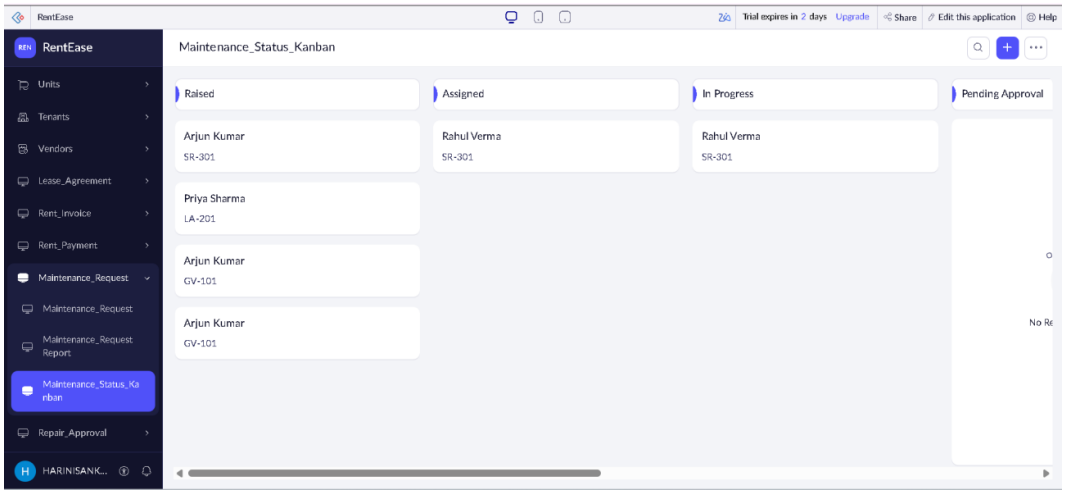


Figure 5.6: Kanban Report: Maintenance_Status_Kanban

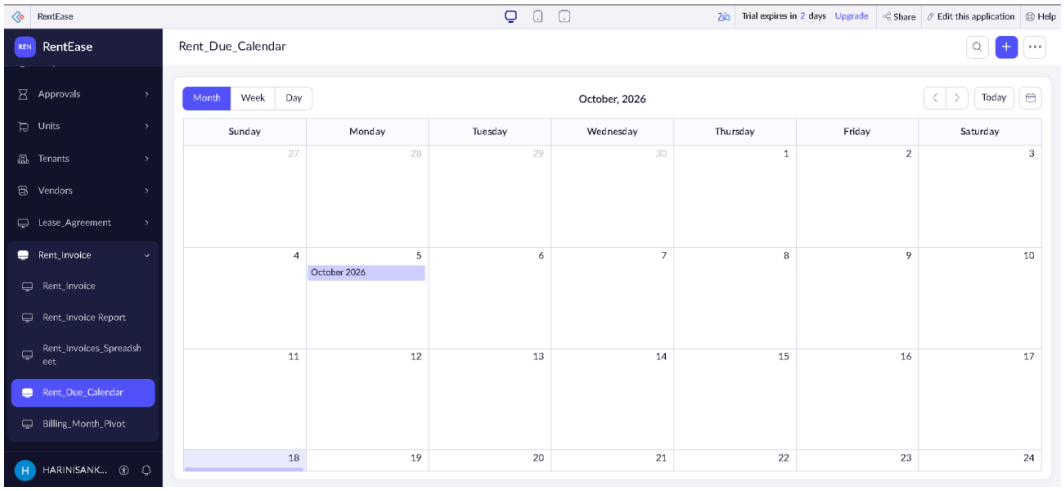


Figure 5.7: Calendar Report: Rent_Due_Calendar

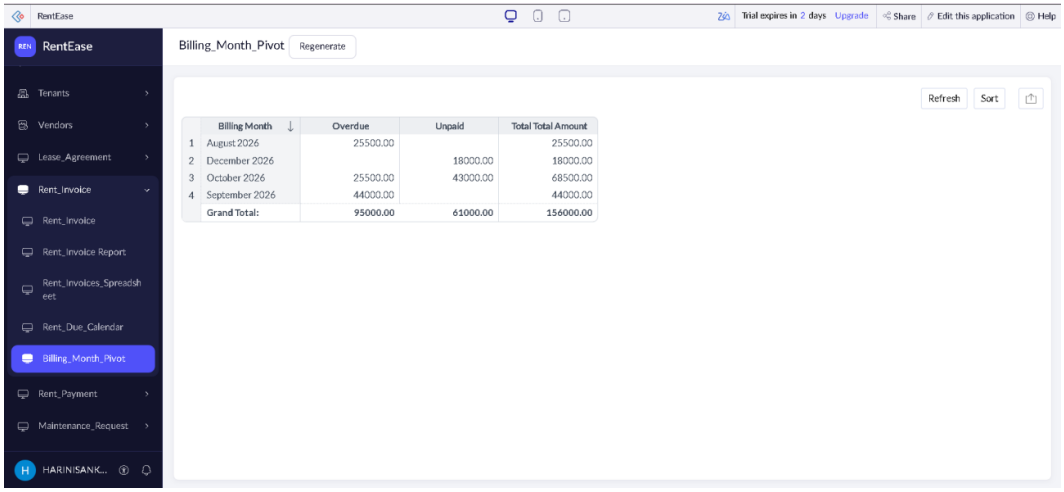


Figure 5.8: Pivot Report: Billing_Month_Pivot

5.4 Pages and Dashboards

Two pages were built. RentEase_Dashboard shows number cards (total billed, rent collected, late fees charged and total leases), a status row (occupied units, vacant units, overdue invoices and open repairs), four buttons that open forms and a report, and two charts. It was generated with Zia's Build Agent and then checked against the data. RentEase_Command_Center was built in the Page Builder with a count panel for occupied units, an occupancy gauge, a revenue chart, a tenant search and a Send Rent Reminders button that runs the function sendRentReminders after confirmation.

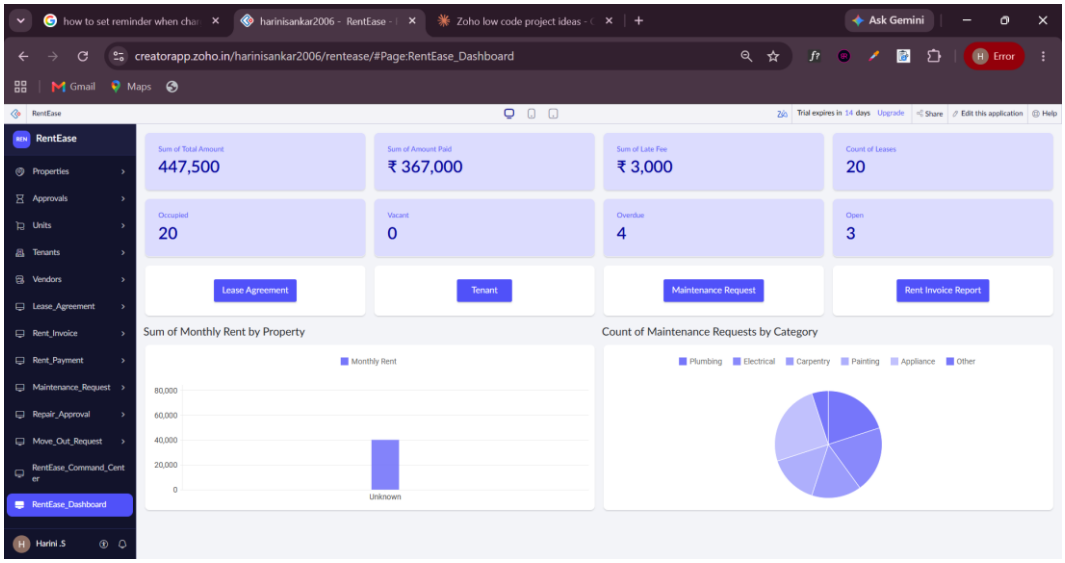


Figure 5.9: RentEase_Dashboard

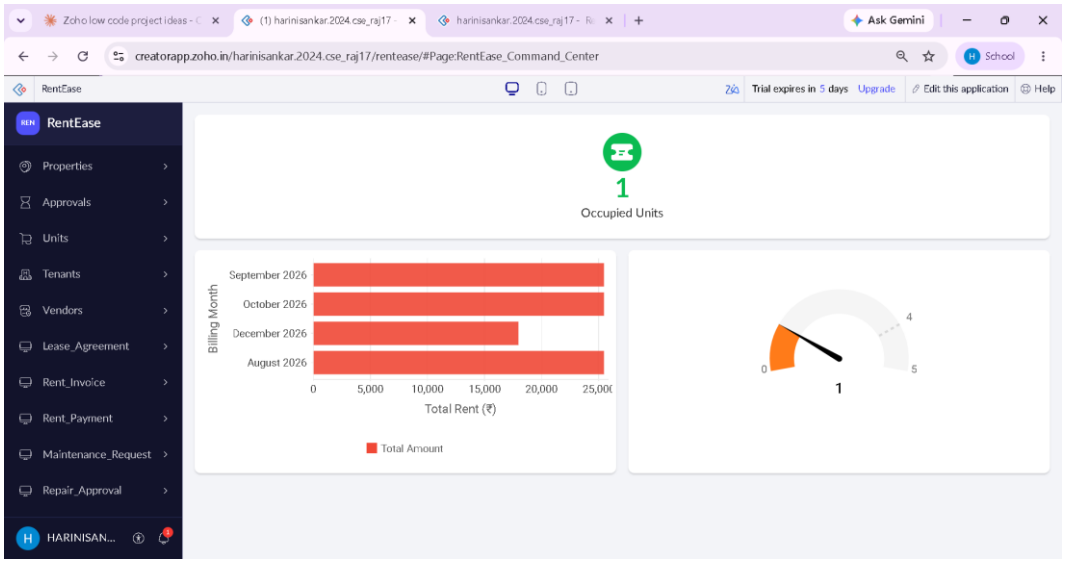


Figure 5.10: RentEase_Command_Center with Panel, Chart and Gauge

5.5 Deluge Scripting

Table 5.2: Deluge Scripts Implemented

Script ID	Location / Trigger	Purpose	Deluge Concepts Used
DS-01	Lease_Agreement → On user input of Unit	Fill Monthly Rent from the selected unit	fetch with criteria, null check, if
DS-02	Lease_Agreement → On success	Build Lease_Reference as LEASE-2026-n	fetch by ID, string concatenation
DS-03	Lease_Agreement → On success	Set unit to Occupied and email the tenant	fetch, update field, sendmail
DS-04	Rent_Invoice → On user input of Lease	Fill Rent Amount from the lease	fetch, null check

Script ID	Location / Trigger	Purpose	Deluge Concepts Used
DS-05	Rent_Payment → On user input of Invoice	Fill Amount Paid from the invoice	fetch, null check
DS-06	Rent_Payment → On success	Mark invoice Paid and call the receipt function	fetch, update, function call
DS-07	Move_Out_Request → On validate	Total the deductions and calculate the refund	for each over a sub-form, arithmetic
DS-08	Maintenance_Request → On success	Email the manager about a new request	sendmail, string concatenation
DS-09	Function generateReceipt(invoiceId)	Email a rent receipt for an invoice	function parameter, null check, return, sendmail
DS-10	Function sendRentReminders()	Email tenants with unpaid invoices	for each with criteria, sendmail
DS-11	Schedule Daily_Overdue_Rent	Mark overdue, add late fee, email tenant	criteria fetch, date comparison, for each, sendmail
DS-12	Schedule Weekly_Maintenance_Digest	Email a digest of open repairs	string building, for each, sendmail
DS-13	Schedule Monthly_Invoice_Generator	Create an invoice for each active lease	insert into, addDay, toString
DS-14	Blueprint → After tab of each transition	Set the Status field for the new stage	field assignment

Code 5.1: DS-03 On_Lease_Created (form workflow)

```
// DS-03 | Lease_Agreement | On success
// Purpose: Mark the unit occupied and email the tenant
u = Unit[ID == input.Unit];
u.Status = "Occupied";
t = Tenant[ID == input.Tenant];
sendmail
[
    from : zoho.adminuserid
    to : t.Email
    subject : "Welcome to RentEase - Lease " + input.Lease_No
    message : "Dear tenant, your lease is confirmed. Monthly rent: " +
input.Monthly_Rent
]
```

Code 5.2: DS-11 Daily_Overdue_Rent (schedule)

```
// DS-11 | Schedule Daily_Overdue_Rent
// Purpose: Mark overdue invoices, add the late fee and email the
tenant
for each inv in Rent_Invoice[Status == "Unpaid" && Due_Date <
zoho.currentdate]
{
    inv.Status = "Overdue";
    inv.Late_Fee = 500;
    sendmail
    [
        from : zoho.adminuserid
        to : inv.Lease_Agreement.Tenant.Email
        subject : "Rent overdue"
        message : "Your rent for " + inv.Billing_Month + " is overdue.
A late fee of 500 has been added."
    ]
}
```

DS-01, DS-04 and DS-05 fill values the moment a lookup is chosen, so rent and amounts are never retyped. DS-02, DS-03, DS-06 and DS-08 run after a record is saved because the lease number, the email and the status change must follow a successful save. DS-07 runs on validate so the saved record already holds the refund. DS-09 and DS-10 are functions: DS-09 is called from DS-06 and DS-10 from the dashboard button, so the receipt and reminder logic are written once. DS-11 to DS-13 run on schedules and use a criteria fetch so only the needed records are read. DS-14 keeps the Status field in step with the blueprint stage. In testing, each script produced the expected result in the reports or by email (Chapter 6).

5.6 Workflows and Automation

Fourteen automations were built: eight form workflows, three schedules, one blueprint and two approvals. Table 5.3 lists them with their triggers and script references. The two functions called by them are listed in Table 5.2.

Table 5.3: Workflows Implemented

Workflow Name	Type	Trigger / Event	Condition	Action(s)	Script Ref.
Fill_Rent_From_Unit	Form Workflow	Created or Edited; user input of Unit	Unit selected	Fill monthly rent	DS-01
Set_Lease_Reference	Form Workflow	Created; on success	—	Set Lease_Reference	DS-02
On_Lease_Created	Form Workflow	Created; on success	—	Unit Occupied; welcome email	DS-03

Workflow Name	Type	Trigger / Event	Condition	Action(s)	Script Ref.
Fill_Rent_From_Lease	Form Workflow	Created or Edited; user input of Lease	Lease selected	Fill rent amount	DS-04
Fill_Amount_From_Invoice	Form Workflow	Created or Edited; user input of Invoice	Invoice selected	Fill amount paid	DS-05
On_Payment_Mark_Paid	Form Workflow	Created; on success	—	Invoice Paid; call generateReceipt	DS-06, DS-09
Calculate_Refund	Form Workflow	Created or Edited; on validate	—	Total deductions and refund	DS-07
Notify_Manager_On_Request	Form Workflow	Created; on success	—	Email the manager	DS-08
Daily_Overdue_Rent	Schedule	Daily	Status Unpaid and due date before today	Overdue, late fee 500, email	DS-11
Weekly_Maintenance_Digest	Schedule	Weekly (Monday)	Status not Closed	Email digest	DS-12
Monthly_Invoice_Generator	Schedule	Monthly (1st)	Lease Status Active	Create invoice	DS-13
Maintenance_Lifecycle_Blueprint	Blueprint	Every Maintenance_Request	Estimated_Cost above 5000 shows Send for Approval; 5000 or less shows Complete	Move through seven stages; set Status	DS-14
Repair_Cost_Approval	Approval	On add of Repair_Approval	Quoted_Amount above 5000	Two approval levels; set Approval_Status	—
Deposit_Refund_Approval	Approval	On add of Move_Out_Request	Always	Approval; set Status	—

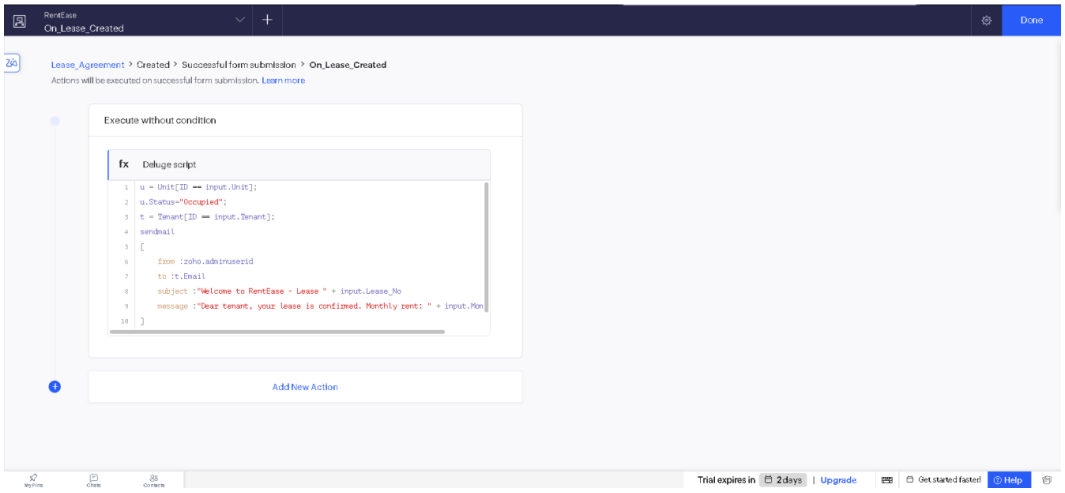


Figure 5.11: Form Workflow On_Lease_Created and its Script

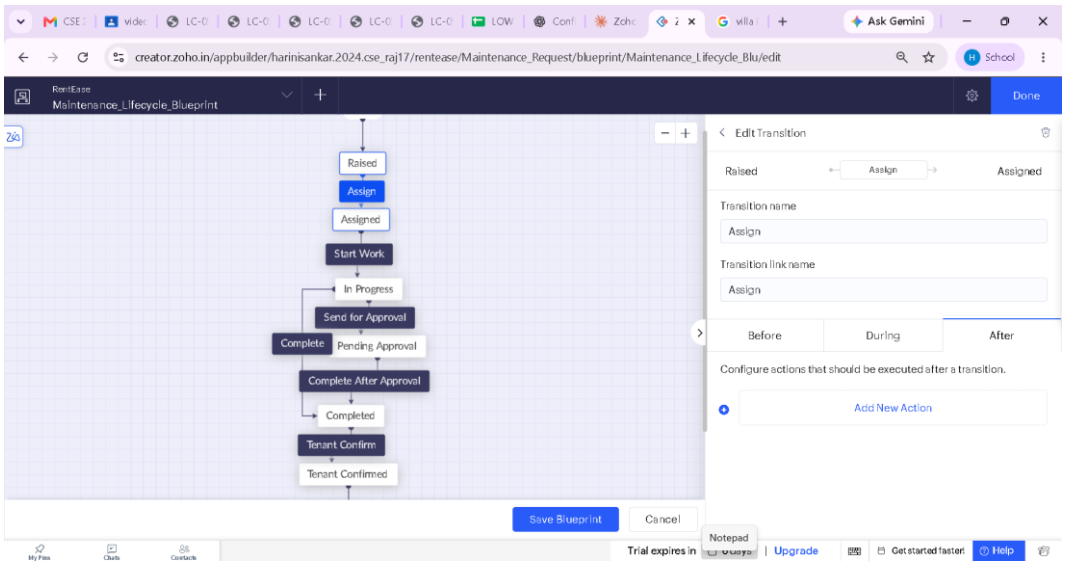


Figure 5.12: Blueprint Canvas: Maintenance_Lifecycle_Blueprint

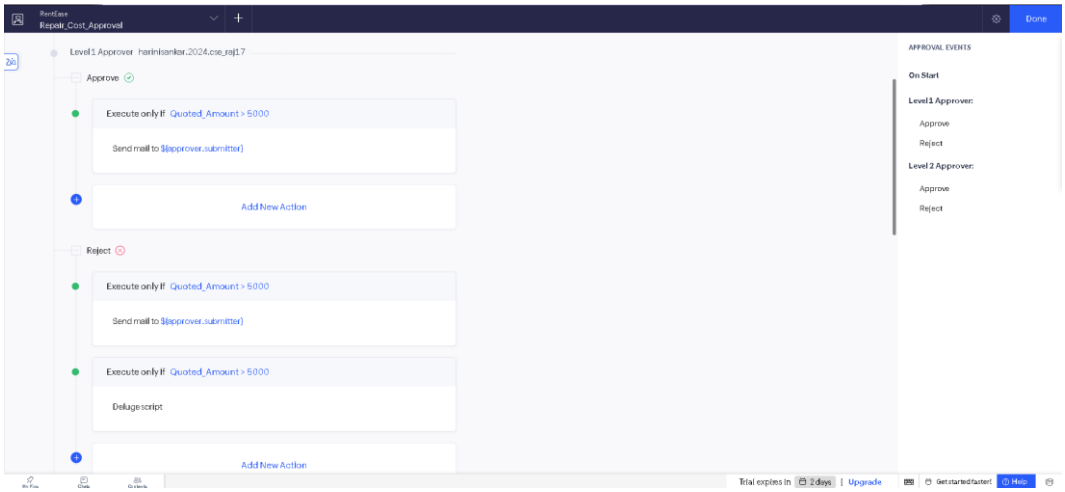


Figure 5.13: Approval Levels and Actions: Repair_Cost_Approval

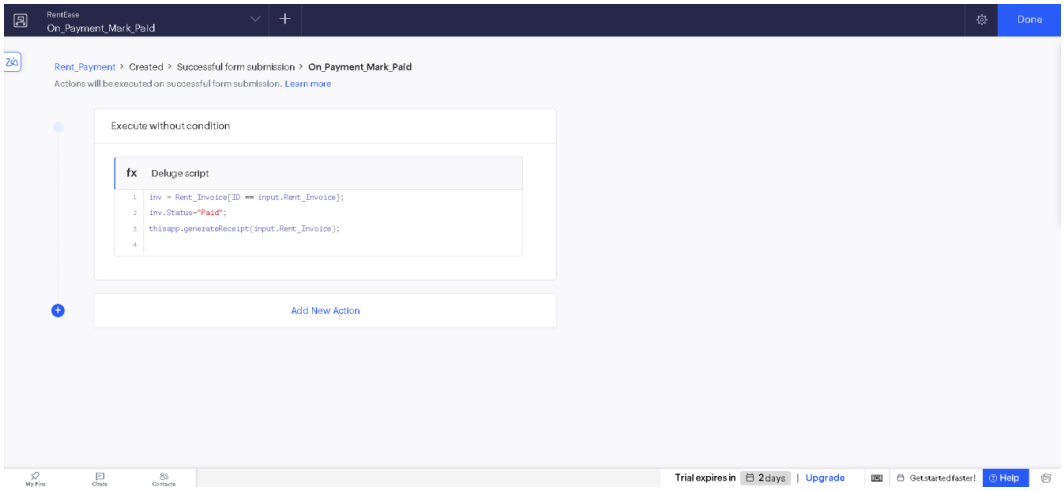


Figure 5.14: Form Workflow On_Payment_Mark_Paid Calling the Function generateReceipt

5.7 Integrations

No integration with another Zoho service or an external API was implemented, because every requirement of RentEase is met inside Zoho Creator. Notifications are sent with the built-in sendmail task of Creator (scripts DS-03, DS-06, DS-08, DS-10, DS-11 and DS-12), which needs no connection. Possible integrations, such as a payment gateway or an SMS service, are listed as future enhancements in Section 9.2.

5.8 User Management and Role-Based Access

Tenants are external users, so access is controlled through a portal permission set named Tenant_Profile. It gives Access and View on the Lease_Agreement, Rent_Invoice, Rent_Payment, Maintenance_Request and Move_Out_Request reports, and no access to Property, Unit, Tenant, Vendor and Repair_Approval. Edit and Delete are off, and the View all and Modify all options were left off. The Lease_Reference field is visible to the administrator only. Separate internal roles for the manager and the owner were not created: the administrator account acts as both and is the approver in both approvals. Table 4.12 shows the intended matrix, and creating the internal roles is listed as a future enhancement.

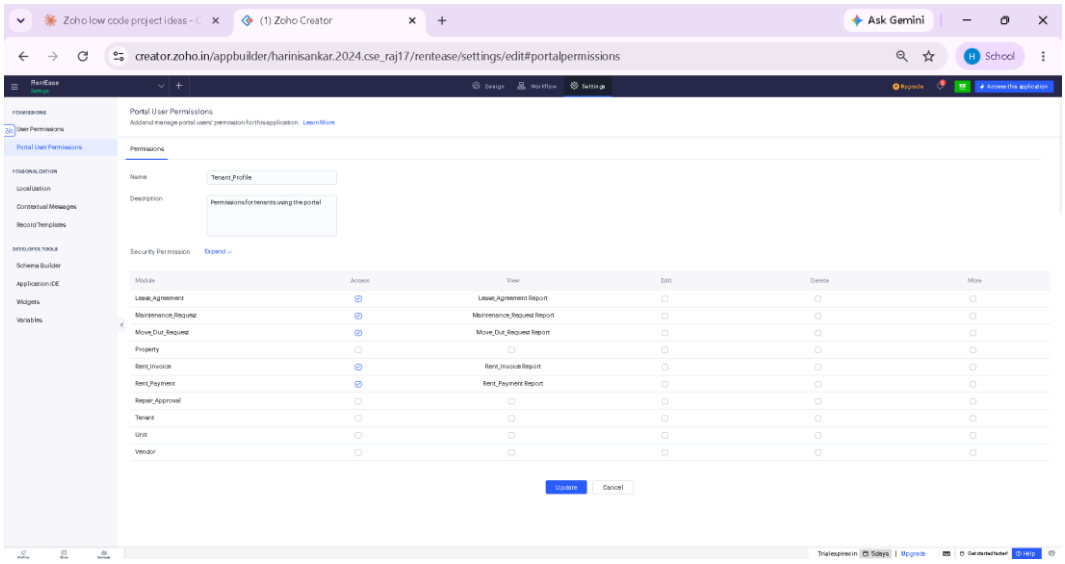


Figure 5.15: Tenant_Profile Permission Set for Portal Users

5.9 AI and Analytics Features

Zia's Build Agent was used to create the RentEase_Dashboard page from a plain-language description. It produced the cards, status row, buttons and charts, and the generated page was checked against the data. The Agent paused at its tool-usage limit several times, and some attempts to add elements or change styles failed with errors, so the remaining styling is done in the Page Builder. In an earlier experiment Zia was also used to build a small separate application. Analytics in RentEase use the built-in chart, pivot, summary and panel elements: the dashboard cards and charts read live aggregates (sum and count) from the forms, so no external analytics service was needed.

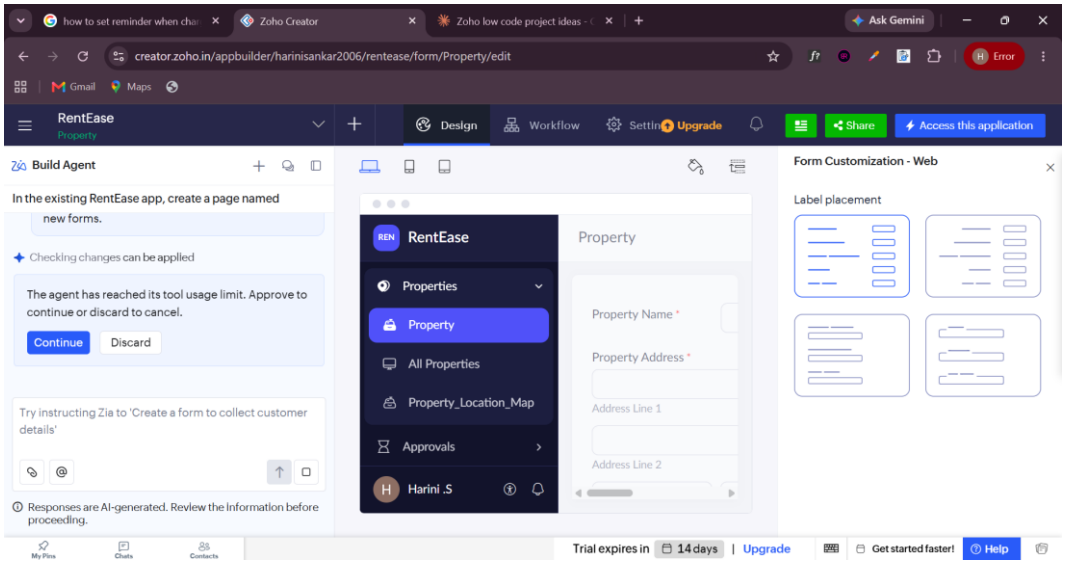


Figure 5.16: Zia Build Agent Used for the Dashboard Page

5.10 Pro-Code Extension

No pro-code extension was built. Every requirement was met with forms, workflows, Deluge scripts, reports and pages. A widget or a Catalyst function would only be justified for logic that Deluge cannot handle, such as heavy computation or a custom UI component, and RentEase has no such need. Section 8.5 places the project on the no-code, low-code and pro-code spectrum.

5.11 Mobile and Portal Access

A customer portal was created from the Creator home page with the Tenant_Profile permission set as the default. A tenant is invited from the Portal User tab, receives an email, sets a password and signs in at the portal address. The test tenant saw only the permitted menu and raised a maintenance request that reached the manager. Records created by the administrator did not appear in the tenant's reports, so record-level filtering is still to be configured. The Creator mobile app was not tested.

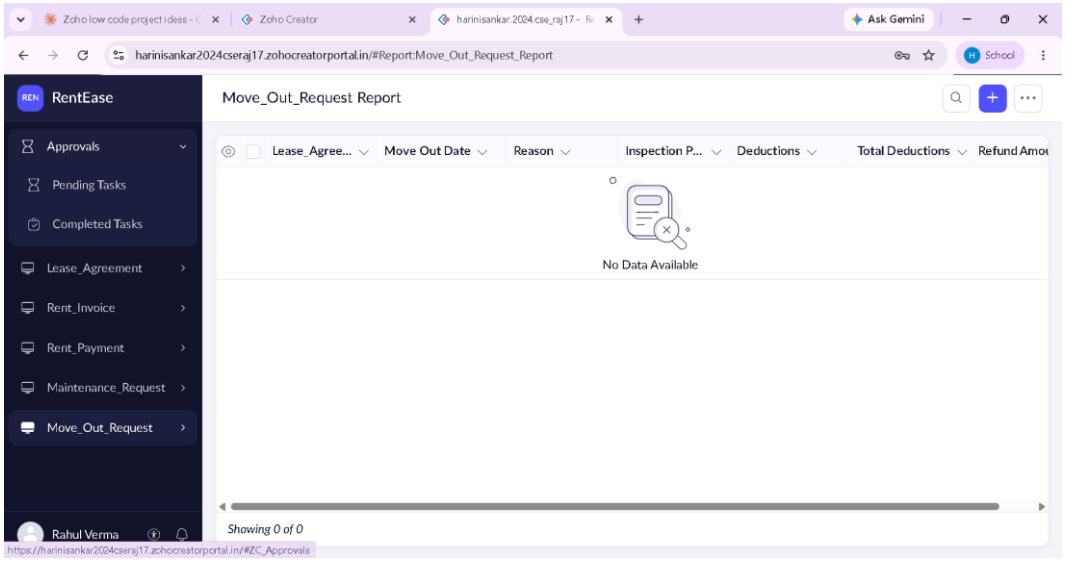


Figure 5.17: Tenant Portal after Sign-in

CHAPTER 6
TESTING AND DEBUGGING

6.1 Testing Strategy

Testing was done in the single live environment with sample data. Each form and workflow was tested as it was built: the field validations were tried with missing values, each workflow was triggered by creating a record and its effect was checked in the report or by email, the blueprint and approvals were run end to end with new records, and the tenant portal was tested with an invited tenant. User acceptance feedback is recorded in Section 6.5. Test cases are traced to the functional requirements of Table 3.2.

6.2 Test Cases

Table 6.1: Test Cases

TC ID	FR Ref.	Test Description	Input / Steps	Expected Result	Actual Result	Status
TC-01	FR-02	Create a lease	Select tenant and unit, enter dates and deposit, sign, submit	Record saved; lease number, reference and QR created	Lease No 2 with reference LEASE-2026-2, signature and QR shown in the report	Pass
TC-02	FR-02	Rent auto-fill from the unit	Select a unit on the lease form	Monthly Rent filled from the unit	Rent filled (Rs 18,000 for the unit used)	Pass
TC-03	FR-02	Unit occupancy and welcome email	Submit a lease for a vacant unit	Unit becomes Occupied; welcome email sent	Unit changed to Occupied; email received	Pass
TC-04	FR-03	Mandatory validation on invoice	Submit an invoice with Rent Amount empty	Validation error; record not saved	“This field is mandatory. Enter a value.” shown	Pass
TC-05	FR-03	Invoice rent and total	Select a lease on a new invoice	Rent filled; Total = rent + late fee	Rent filled; Total Amount Rs 25,000.00 for rent 25,000 and fee 0	Pass
TC-06	FR-05	Payment amount auto-fill	Select an invoice on a new payment	Amount Paid filled from the invoice	Amount filled	Pass

TC ID	FR Ref.	Test Description	Input / Steps	Expected Result	Actual Result	Status
TC-07	FR-05	Payment marks invoice paid	Submit a payment	Invoice becomes Paid; receipt emailed	Invoice status Paid; receipt email received	Pass
TC-08	FR-04	Overdue schedule	Run the schedule with unpaid invoices past their due date	Status Overdue; late fee 500; email sent	Invoices show Overdue with a late fee of Rs 500	Pass
TC-09	FR-09	Move-out refund	Add three deduction rows and save	Total deductions and refund calculated	Totals and refund calculated automatically	Pass
TC-10	FR-11	New request notification	Create a maintenance request	Manager receives an email	Email “New maintenance request: Electrical” received	Pass
TC-11	FR-07	Blueprint, cost above 5,000	Request with cost 9,000: Assign, Start Work, then continue	Send for Approval offered; stages completed to Closed	Record moved through all stages to Closed	Pass
TC-12	FR-07	Status follows blueprint stage	Complete a transition and refresh the report	Status column shows the new stage	Failed at first (stayed Raised); passed after DS-14 was added	Pass
TC-13	FR-08	Approval of a large quote	Add a quote of 8,000 and approve both levels in the inbox	Approval Status becomes Approved; email sent	Status Approved; approval email received	Pass
TC-14	FR-08	Rejection of a quote	Add a quote of 12,000 and reject at level 1	Approval Status becomes Rejected	Not yet executed	Not run
TC-15	FR-08	Small quote skips approval	Add a quote of 3,000	No approval item is created	Not yet executed	Not run
TC-16	FR-06	Tenant portal sign-in	Invite a tenant, set a password and sign in	Only the permitted menu is shown	Menu showed the Approvals, Lease_Agreement, Rent_Invoice, Rent_Payment, Maintenance_Request and Move_Out_Request entries	Pass

TC ID	FR Ref.	Test Description	Input / Steps	Expected Result	Actual Result	Status
TC-17	FR-06	Tenant raises a request	Submit a maintenance request in the portal	Record created and visible to the manager	Request submitted and visible in the administrator's report	Pass
TC-18	FR-10	Dashboard figures	Open RentEase_Dashboard	Cards and charts show live data	Showed 20 leases, Rs 3,67,000 collected, 4 overdue invoices and 3 open repairs	Pass

6.3 Debugging Deluge Scripts

Debugging relied on the checks Creator provides. The Deluge editor reports errors when a script is saved, so wrong field link names were found immediately. Each script was then tested by creating a fresh record and checking the result in the report or in the mailbox, because approvals and blueprints apply only to records created after they are enabled. Execution of the email workflows was confirmed by the notification mails received. When a result was wrong, the field link names in Edit mode were compared with the script and corrected.

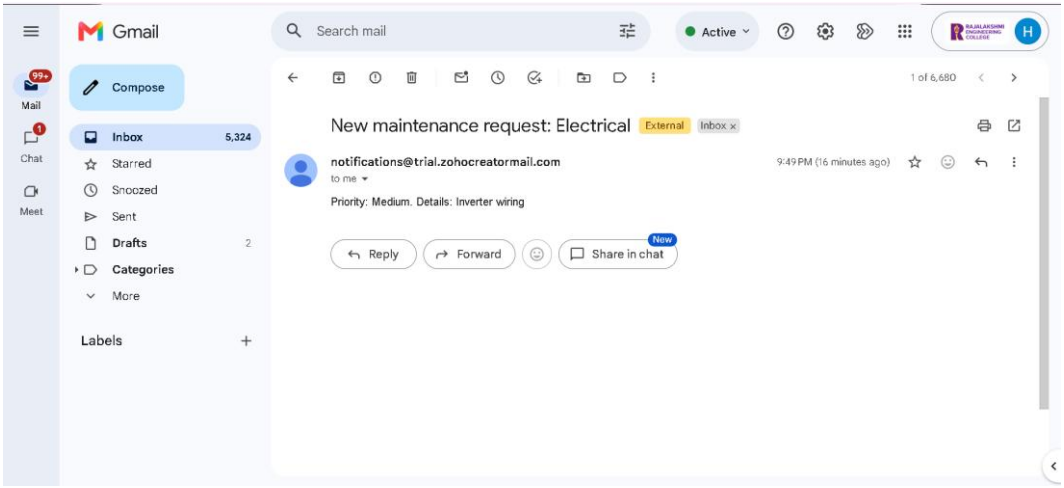


Figure 6.1: Evidence of Workflow Execution: Manager Notification Email

6.4 Defect Log

Table 6.2: Defect Log

Bug ID	Description	Root Cause	Fix Applied	Status
BUG-01	The lookup field on Rent_Invoice and Rent_Payment was named after its parent form, so scripts using Lease or Invoice failed.	Creator takes the form name as the lookup link name.	Scripts changed to the real link names (Lease_Agreement, Rent_Invoice).	Closed
BUG-02	The Auto Number field has no prefix option, so LEASE-2026- could not be generated.	Feature not available in this version.	Lease_Reference text field filled by workflow DS-02.	Closed
BUG-03	Number fields have no minimum or maximum value.	Limitation of the field type.	Accepted; an On Validate script is the workaround.	Open (accepted)
BUG-04	The blueprint moved the record but the Status column stayed at Raised.	Blueprint stages are separate from the Status field.	After-tab Deluge script sets Status on every transition (DS-14).	Closed
BUG-05	Existing records showed no blueprint button and no approval item.	Blueprints and approvals apply only to new records.	Tested with newly created records.	Closed
BUG-06	The Zia Build Agent stopped at its tool limit and Apply failed with page errors.	Agent limit and a page-update error.	Page elements and styles were corrected in the Page Builder.	Closed
BUG-07	The portal user could not be added with the administrator’s own email.	The address already existed as a Creator user.	A tenant with a different email was invited.	Closed
BUG-08	Tenant reports looked empty in the portal.	Sample records were created by the administrator and no record filter was set.	Record-level criteria are planned (Section 9.2).	Open

6.5 User Acceptance Testing

Table 6.3: User Acceptance Feedback

User	Role Tested	Feedback	Action Taken
Rahul Verma	Tenant (Portal User)	Login was successful, the permitted menu was displayed, and the maintenance request could be submitted successfully.	Verified portal access and maintenance request submission. Record-level filtering was noted for future enhancement.
Priya Sharma	Property Manager (Administrator Account)	Lease creation, rent invoice processing, payment updates, notifications and approval workflows worked as expected.	Verified workflow execution, checked reports and email notifications, and corrected the identified defects.

CHAPTER 7
DEPLOYMENT, SECURITY AND GOVERNANCE

7.1 Environments and Deployment Strategy

RentEase is developed and used in a single live environment. The Environments option (Development, Stage, Production) was not enabled, because the project is built and demonstrated by one developer and the option adds publishing steps that were not needed. Changes were therefore tested directly with sample data. For a business deployment, the Stage and Production environments would be enabled so that changes could be tested before they reach real users.

Table 7.1: Deployment Checklist

S.No.	Checklist Item	Done (✓)
1	All test cases executed in Development / Stage	✓
2	Sample/test data removed or isolated	✗ (sample data kept for demonstration)
3	Roles, permissions and users configured in Production	✓
4	Connections authorised in Production	N/A (no connections)
5	Schedules and workflows enabled	✓
6	Application published and URL verified	✓
7	Version / release notes recorded	✗

7.2 Performance Optimisation

Performance was handled at design level. No timing measurements were taken, so Table 7.2 describes the techniques applied and the expected effect, not measured gains. With about 20 records per form, response times were not noticeable.

Table 7.2: Performance Optimisation Results

Optimisation Applied	Component	Before	After	Improvement
Criteria-based fetch in the daily schedule	DS-11	Fetch every invoice and test each in a loop	Single fetch with Status and Due_Date criteria	Fewer records processed (not timed)
Fetch by record ID in the auto-fill scripts	DS-01, DS-04, DS-05	Search the whole form for a match	Direct fetch by ID	Single-record read (not timed)
Panels and charts read aggregates directly	Dashboard	Loop script to total records	Built-in sum and count	No script needed (not timed)

7.3 Security Features

Table 7.3: Security Controls Implemented

Security Control	Where Applied	Threat Mitigated
Portal permission set (Tenant_Profile)	Tenant portal	Tenants reaching internal forms and reports
Portal sign-in by invitation and password	Tenant portal	Anonymous access to tenant data
No duplicate values on Tenant Email	Tenant form	Duplicate or ambiguous portal identities
Admin Only field visibility	Lease_Reference field	Accidental editing of a system value
Two-level approval for large amounts	Repair_Approval, Move_Out_Request	Unapproved spending and refunds

7.4 Governance and Administration

Governance in RentEase rests on features of Creator. Every record keeps the user who added it and the user who last modified it. Approvals keep a decision history in the approvals inbox, and blueprint transitions keep a record of when and by whom each stage change was made. Workflows, schedules and scripts follow a naming convention with underscores, and all scripts are documented in Appendix C. Separate administrator roles, audit reports and environment-based release management are not configured and are noted as future work.

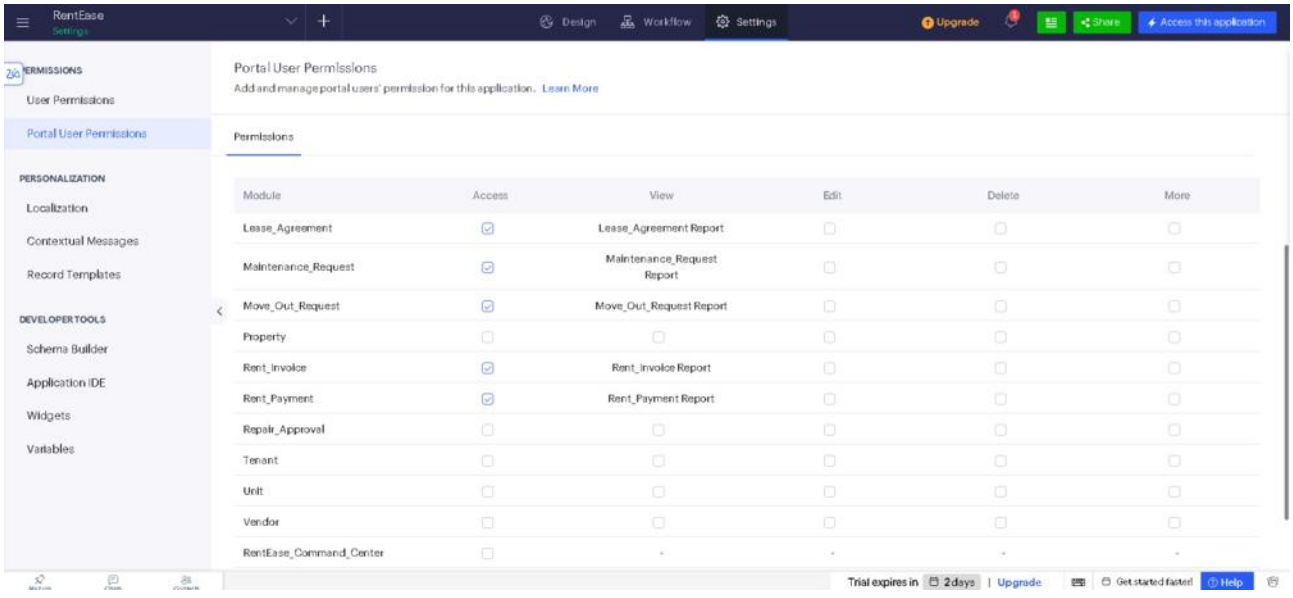


Figure 7.1: Audit Information: Record History and Approvals Inbox

CHAPTER 8
RESULTS AND DISCUSSION

8.1 Application Output

The scenario uses a sample tenant and unit. A manager creates a lease for tenant Rahul Verma and unit SR-301 (monthly rent Rs 18,000, deposit Rs 36,000). The rent fills in, the lease reference and QR code are generated, the unit becomes Occupied and a welcome email is sent (Figure 8.1). An invoice is created for the lease; invoices not paid by the due date are marked Overdue with a late fee of Rs 500, and a saved payment marks an invoice Paid and sends a receipt (Figure 8.2). A repair with a quote above Rs 5,000 goes to the approvals inbox as an approval summary (Figure 8.3), and once approved the Approval Status changes (Figure 8.4).

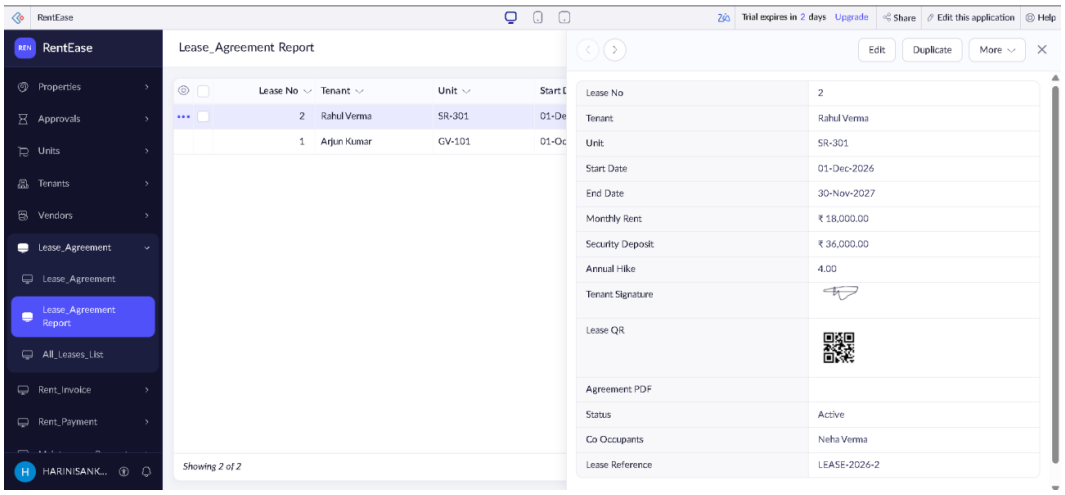


Figure 8.1: Scenario Step 1 – Lease created with lease number, reference and QR code

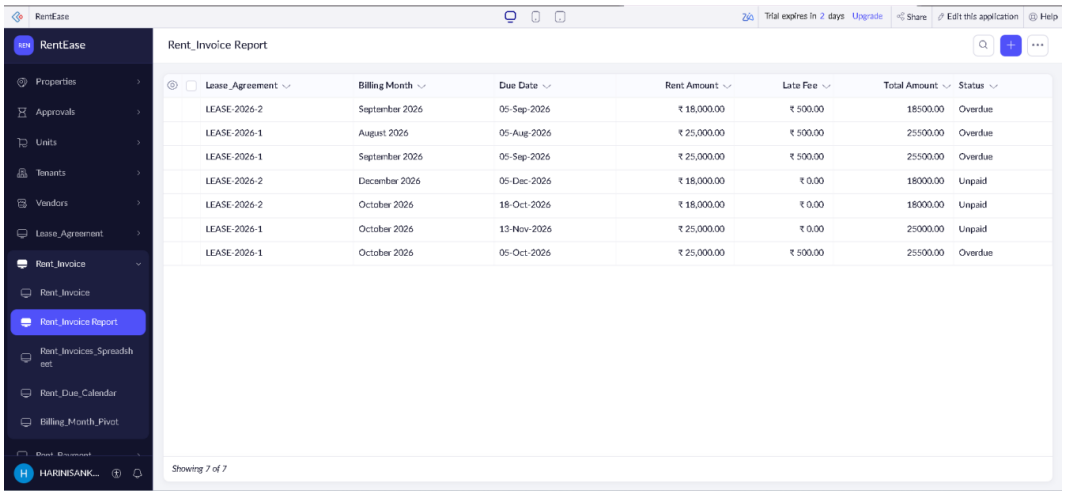


Figure 8.2: Scenario Step 2 – Invoices marked Overdue with the late fee

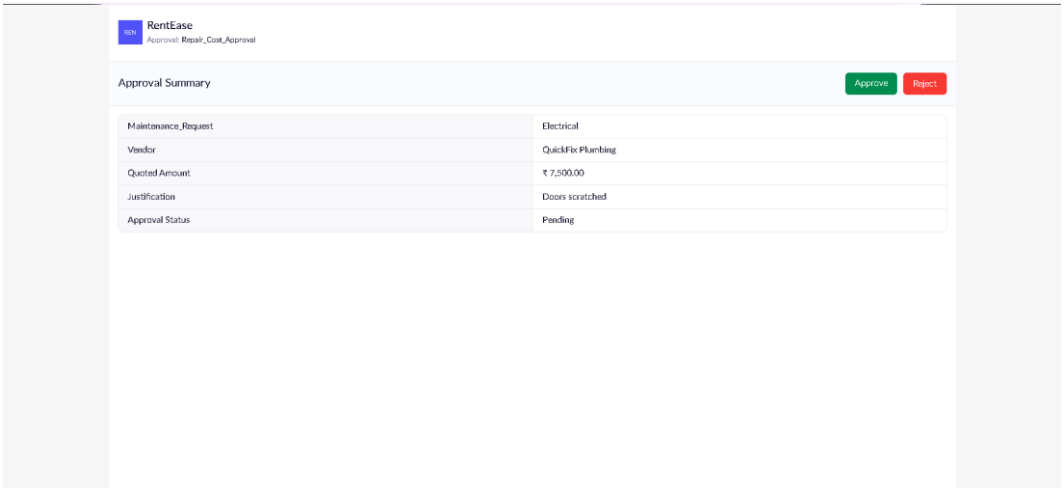


Figure 8.3: Scenario Step 3 – Repair quote waiting in the approval summary

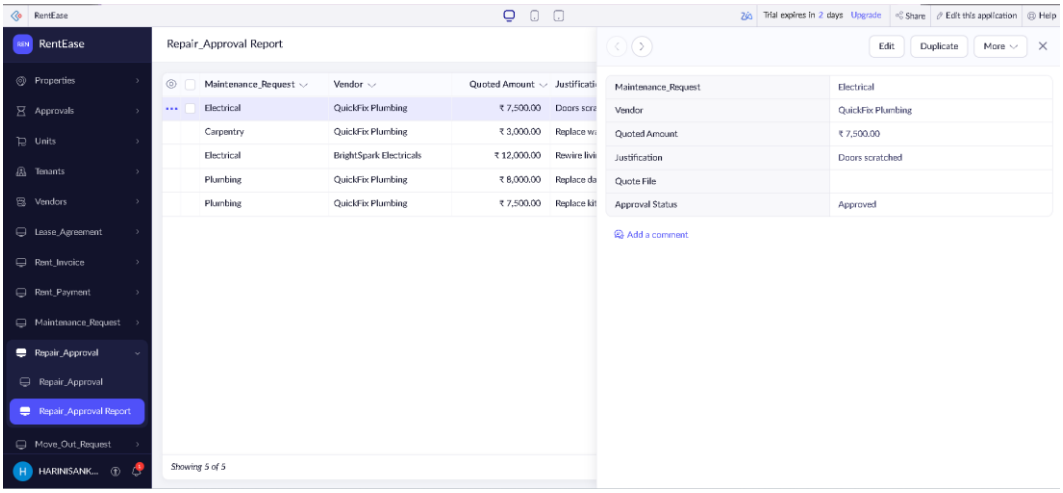


Figure 8.4: Scenario Step 4 – Repair_Approval record after approval

8.2 Achievement of Objectives

Table 8.1: Mapping of Objectives to Outcomes

Objective (Sec. 1.3)	Implemented Through	Outcome / Evidence
To design a data model of ten linked forms	Section 5.2, Table 4.1	Ten forms with lookups and two sub-forms, tested with sample data
To automate rent, referencing, status, receipts, overdue charges and notifications	Sections 5.5 and 5.6	Test cases TC-02 to TC-10 passed
To provide a dashboard and nine report types	Sections 5.3 and 5.4	Nine reports and a dashboard showing live data (TC-18)
To control repairs with a blueprint and route large spends through approvals	Section 5.6	TC-11 to TC-13 passed; TC-14 and TC-15 not yet run

Objective (Sec. 1.3)	Implemented Through	Outcome / Evidence
To give tenants a restricted portal and verify the application	Sections 5.8, 5.11 and Chapter 6	TC-16 and TC-17 passed; record-level filtering still open (BUG-08)

8.3 Comparison with the Existing System

Table 8.2: Existing System versus Proposed Application

Parameter	Existing System	Proposed Application	Improvement
Time to process one request	Several minutes of searching spreadsheets and chats for each lease, invoice or complaint	Records are selected from lookups and key values fill in automatically	Faster and more consistent (qualitative; not timed)
Manual steps	Rent, late fees, receipts and refunds worked out by hand	Rent, status changes, late fees, receipts and refunds are automatic	Fewer manual steps and fewer errors
Report generation	Spreadsheet formulas and manual summaries	Nine report types and a live dashboard	Instant views for each user group
Data accessibility	Held by the owner; tenants must ask	Manager and owner use the application; tenants use the portal	Controlled self-service access

8.4 Limitations of the Low-Code Approach

Several limits were met. Number fields have no minimum or maximum value, so a rule such as a minimum number of units needs a validation script. The Auto Number field has no prefix option, so the LEASE-2026- reference had to be built with a workflow. Blueprint stages do not update a status field unless a script does it. Page styling is limited, and the Zia Build Agent stopped at its usage limit and could not apply some style and element changes, so they were made by hand. Imported records do not run workflows, and record-level filtering for the portal needs extra configuration. Batch and report workflows were not built because the schedules and form workflows already cover the needs of a small portfolio. Dependence on the platform and its subscription is a general risk of low-code.

8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum

On the spectrum, the forms, reports, pages, blueprint, approvals and portal are no-code or low-code configuration done in visual builders. The Deluge scripts (DS-01 to DS-14) are the low-code part, where the business rules are written as short scripts. No pro-code component was needed. The project shows that most of a rental management system can be configured, with a small amount of scripting for the rules, and that pro-code would be considered only for needs the platform cannot meet.

CHAPTER 9
CONCLUSION AND FUTURE ENHANCEMENTS

9.1 Conclusion

RentEase brings leases, rent, repairs and deposit refunds of a small property owner into one Zoho Creator application. Ten linked forms, fourteen automations, two functions, nine reports, a dashboard and a tenant portal were built and tested with sample data. Eighteen test cases were defined; sixteen passed and two are yet to be run. The project met its objectives and showed both the speed of low-code development and its limits. Gaps remain: a real payment gateway, separate internal roles, record-level portal filtering and the batch and report workflows.

9.2 Future Enhancements

- Integrate a payment gateway so that tenants can pay rent online, and reconcile payments automatically.
- Create separate Property Manager, Owner and Finance roles and set the approvers to roles instead of one user.
- Add record-level criteria so that each tenant sees only his or her own lease, invoices and requests in the portal.
- Add a batch workflow for monthly overdue reminders and a report workflow that updates unpaid invoices when the rent changes.
- Send SMS or WhatsApp reminders and use Zoho Analytics or Zia for predicting late payers.

9.3 Learning Outcomes and Career Relevance

The project gave practice in the full Zoho Creator cycle: data modelling with lookups and sub-forms, Deluge scripting, workflows, schedules, blueprints, approvals, portals, reports and dashboards, together with testing and defect handling. These skills apply to roles such as low-code developer, business analyst and solution consultant, and to citizen development in any department. Working through the platform’s limits also built judgement about when low-code is the right choice and when pro-code is needed.

REFERENCES

[1] W. Rosam and O. Kalnichevski, *Low-Code Development: A Practical Guide*. New Delhi, India: BPB Publications.

[2] Zoho Corporation, "Zoho Creator Help Documentation." [Online]. Available: https://www.zoho.com/creator/help/ (Accessed: 18 October 2026).

[3] Zoho Corporation, "Deluge Script Reference Guide." [Online]. Available: https://www.zoho.com/deluge/ (Accessed: 18 October 2026).

[4] Zoho Corporation, "Zoho Creator Learning Centre." [Online]. Available: https://www.zoho.com/creator/resources/ (Accessed: 18 October 2026).

[5] Gartner, Inc., "Magic Quadrant for Enterprise Low-Code Application Platforms," *Gartner Research*, 2023.

[6] A. Sahay, A. Indamutsa, D. Di Ruscio and A. Pierantonio, "Supporting the understanding and comparison of low-code development platforms," in Proc. 46th Euromicro Conf. Software Engineering and Advanced Applications (SEAA), 2020.

[7] Y. Luo, P. Liang, C. Wang, M. Shahin and J. Zhan, "Characteristics and challenges of low-code development: The practitioners' perspective," in Proc. ACM/IEEE Int. Symp. Empirical Software Engineering and Measurement (ESEM), 2021.

APPENDIX A – ZOHO CREATOR FEATURE COVERAGE CHECKLIST

Table A.1: Feature Coverage Checklist

Unit	Zoho Creator Feature	Done	Where in the Application	Report Section	Figure / Code No.
I	Application created & sections/menu organised	Y	RentEase application with left menu of forms, reports and pages	5.1	Figure 5.1
II	Forms with ≥ 8 distinct field types	Y	10 forms using Name, Email, Address, Phone, Single Line, Multi Line, Number, Date, Drop Down, Radio, Multi Select, Decision Box, Image, Currency, Percent, Decimal, Date-Time, SubForm, Lookup, Signature, File Upload, Video, Auto Number, Formula, QR	5.2	Figure 5.2, Figure 5.3
II	Lookup field (relationship between forms)	Y	Unit.Property, Lease_Agreement.Tenant and Unit, Rent_Invoice, Rent_Payment and other lookups	5.2	Figure 5.4
II	Subform (parent–child data)	Y	Co_Occupants (Lease_Agreement) and Deductions (Move_Out_Request)	5.2	Figure 5.5
II	Field validations / layout rules	Y	Mandatory fields, no duplicate Tenant Email, initial values, form sections	5.2	Table 4.6
II	List report with filter, grouping, custom action	Partial	All_Leases_List with search; no custom action built	5.3	Table 5.1
II	Kanban / Calendar / Timeline / Map report	Y	Kanban, Calendar, Timeline and Map reports	5.3	Figure 5.6, Figure 5.7
II	Pivot table / chart report	Y	Billing_Month_Pivot and Revenue_By_Property_Chart	5.3	Figure 5.8
II	Page / dashboard built with UI Builder	Y	RentEase_Dashboard and RentEase_Command_Center	5.4	Figure 5.9
II	Deluge – form events (validate / success / field actions)	Y	DS-01 to DS-08 (on user input, on validate, on success)	5.5	Code 5.1

Unit	Zoho Creator Feature	Done	Where in the Application	Report Section	Figure / Code No.
II	Deluge – functions, fetch, loops, collections	Partial	Functions, fetch and loops used; lists and maps (collections) not used	5.5	Code 5.2
II	Email / in-app notifications	Y	sendmail in DS-03, DS-06, DS-08, DS-10, DS-11 and DS-12	5.5, 6.3	Figure 6.1
II	Form workflow	Y	Eight form workflows	5.6	Figure 5.11
II	Scheduled workflow	Y	Daily_Overdue_Rent, Weekly_Maintenance_Digest, Monthly_Invoice_Generator	5.6	Table 5.3
II	Approval process	Y	Repair_Cost_Approval and Deposit_Refund_Approval	5.6	Figure 5.13
II	Blueprint	Y	Maintenance_Lifecycle_Blueprint	5.6	Figure 5.12
II	Batch / report workflow	N	Not built; reasons in Section 8.4	8.4	—
II	Testing & debugging evidence	Y	18 test cases and a defect log	6.2–6.4	Table 6.1, Table 6.2
III	Integration with a Zoho service	N	No integration with another Zoho service	5.7	—
III	Connection / REST API call (invokeurl)	N	No external connection or invokeurl call	5.7	—
III	Webhook or Creator REST API	N	No webhook or Creator REST API call	5.7	—
III	Users, roles and permission profiles	Partial	Tenant_Profile portal permission set; internal roles not created	5.8	Figure 5.15
III	Record-level access criteria / customer portal	Partial	Tenant portal built; record-level criteria still to be set	5.11	Figure 5.17
III	Zia / AI feature	Y	Zia Build Agent generated the dashboard page	5.9	Figure 5.16
III	Analytics (charts / Zoho Analytics)	Y	Charts, pivot, summary and dashboard cards	5.3, 5.4	Figure 5.9
IV	Performance optimisation applied & measured	Partial	Techniques applied; no timings measured	7.2	Table 7.2

Unit	Zoho Creator Feature	Done	Where in the Application	Report Section	Figure / Code No.
IV	Deployment across environments	N	Single live environment; Stage and Production not used	7.1	—
IV	Security controls (encryption, OAuth, permissions)	Partial	Permission set, portal sign-in, no-duplicate rule, Admin Only field, approvals	7.3	Table 7.3
IV	Governance (audit trail, administration)	Partial	Added and modified user, approval history, blueprint history	7.4	Figure 7.1
V	Pro-code extension (Catalyst / widgets / custom functions)	N	No pro-code extension	5.10	—
V	Low-code limitations & spectrum analysis	Y	Limitations and spectrum discussed	8.4, 8.5	—
—	Mobile app / portal access	Partial	Tenant portal used; Creator mobile app not tested	5.11	Figure 5.17

APPENDIX B – COURSE OUTCOME MAPPING

Table B.1: Mapping of Course Outcomes to Report Chapters

CO	Course Outcome Statement	Evidenced In	Self-Assessed Level	Examiner Level
CO1	Explain the principles, architecture, and business value of low-code application development using Zoho Creator.	Ch. 1, 2		
CO2	Develop business applications using Zoho Creator forms, reports, workflows, and Deluge scripting.	Ch. 4, 5 (5.1–5.6), 6		
CO3	Integrate Zoho Creator applications with Zoho services and external systems using APIs, connections, and role-based security.	Ch. 4.6, 5.7–5.9		
CO4	Deploy, secure, and manage Zoho Creator applications across development and production environments following governance best practices.	Ch. 7		
CO5	Evaluate low-code development strategies, AI-assisted application development, and the transition from citizen developer to professional developer.	Ch. 5.9, 5.10, 8.4, 8.5, 9.3		

APPENDIX C – COMPLETE DELUGE SOURCE CODE

C.1 Lease_Agreement – Fill_Rent_From_Unit

```
// DS-01 | Lease_Agreement | On user input of Unit
// Purpose: Fill Monthly Rent from the selected unit
if(input.Unit != null)
{
    input.Monthly_Rent = Unit[ID == input.Unit].Monthly_Rent;
}
```

C.2 Lease_Agreement – Set_Lease_Reference

```
// DS-02 | Lease_Agreement | On success
// Purpose: Build the lease reference from the lease number
rec = Lease_Agreement[ID == input.ID];
rec.Lease_Reference = "LEASE-2026-" + input.Lease_No;
```

C.3 Lease_Agreement – On_Lease_Created

```
// DS-03 | Lease_Agreement | On success
// Purpose: Mark the unit occupied and email the tenant
u = Unit[ID == input.Unit];
u.Status = "Occupied";
t = Tenant[ID == input.Tenant];
sendmail
[
    from : zoho.adminuserid
    to : t.Email
    subject : "Welcome to RentEase - Lease " + input.Lease_No
    message : "Dear tenant, your lease is confirmed. Monthly rent: " +
input.Monthly_Rent
]
```

C.4 Rent_Invoice – Fill_Rent_From_Lease

```
// DS-04 | Rent_Invoice | On user input of Lease_Agreement
// Purpose: Fill Rent Amount from the lease
if(input.Lease_Agreement != null)
{
    input.Rent_Amount = Lease_Agreement[ID ==
input.Lease_Agreement].Monthly_Rent;
}
```

C.5 Rent_Payment – Fill_Amount_From_Invoice

```
// DS-05 | Rent_Payment | On user input of Rent_Invoice
// Purpose: Fill Amount Paid from the invoice
if(input.Rent_Invoice != null)
{
    input.Amount_Paid = Rent_Invoice[ID ==
input.Rent_Invoice].Total_Amount;
}
```

C.6 Rent_Payment – On_Payment_Mark_Paid

```
// DS-06 | Rent_Payment | On success
// Purpose: Mark the invoice paid and send the receipt
inv = Rent_Invoice[ID == input.Rent_Invoice];
inv.Status = "Paid";
thisapp.generateReceipt(input.Rent_Invoice);
```

C.7 Move_Out_Request – Calculate_Refund

```
// DS-07 | Move_Out_Request | On validate
// Purpose: Total the deductions and calculate the refund
total = 0;
for each row in input.Deductions
{
    total = total + row.Amount;
}
input.Total_Deductions = total;
input.Refund_Amount = Lease_Agreement[ID ==
input.Lease_Agreement].Security_Deposit - total;
```

C.8 Maintenance_Request – Notify_Manager_On_Request

```
// DS-08 | Maintenance_Request | On success
// Purpose: Email the manager about a new request
sendmail
[
    from : zoho.adminuserid
    to : zoho.adminuserid
    subject : "New maintenance request: " + input.Category
    message : "Priority: " + input.Priority + ". Details: " +
input.Description
]
```

C.9 Function – generateReceipt

```
// DS-09 | Function generateReceipt(int invoiceId)
// Purpose: Email a rent receipt for an invoice
inv = Rent_Invoice[ID == invoiceId];
if(inv.Total_Amount == null)
{
    info "Invoice incomplete";
    return;
}
sendmail
[
    from : zoho.adminuserid
    to : inv.Lease_Agreement.Tenant.Email
    subject : "Rent Receipt - " + inv.Billing_Month
    message : "Payment received for " + inv.Billing_Month + ". Amount:
" + inv.Total_Amount
]
```

C.10 Function – sendRentReminders

```
// DS-10 | Function sendRentReminders()
// Purpose: Email every tenant with an unpaid invoice
for each inv in Rent_Invoice[Status == "Unpaid"]
{
    sendmail
    [
        from : zoho.adminuserid
        to : inv.Lease_Agreement.Tenant.Email
        subject : "Rent reminder"
        message : "Your rent for " + inv.Billing_Month + " is pending.
Please pay by " + inv.Due_Date
    ]
}
```

C.11 Schedule – Daily_Overdue_Rent

```
// DS-11 | Schedule Daily_Overdue_Rent
// Purpose: Mark overdue invoices, add the late fee and email the
tenant
for each inv in Rent_Invoice[Status == "Unpaid" && Due_Date <
zoho.currentdate]
{
    inv.Status = "Overdue";
    inv.Late_Fee = 500;
    sendmail
    [
        from : zoho.adminuserid
        to : inv.Lease_Agreement.Tenant.Email
        subject : "Rent overdue"
        message : "Your rent for " + inv.Billing_Month + " is overdue.
A late fee of 500 has been added."
    ]
}
```

C.12 Schedule – Weekly_Maintenance_Digest

```
// DS-12 | Schedule Weekly_Maintenance_Digest
// Purpose: Email a digest of open repair requests
body = "";
for each m in Maintenance_Request[Status != "Closed"]
{
    body = body + m.Category + " | " + m.Priority + " | " + m.Status +
"\n";
}
sendmail
[
    from : zoho.adminuserid
    to : zoho.adminuserid
    subject : "Weekly open maintenance requests"
    message : body
]
```

C.13 Schedule – Monthly_Invoice_Generator

```
// DS-13 | Schedule Monthly_Invoice_Generator
// Purpose: Create an invoice for every active lease
for each l in Lease_Agreement[Status == "Active"]
{
    insert into Rent_Invoice
    [
        Added_User = zoho.loginuser
        Lease_Agreement = l.ID
        Billing_Month = zoho.currentdate.toString("MMMM yyyy")
        Due_Date = zoho.currentdate.addDay(4)
        Rent_Amount = l.Monthly_Rent
        Late_Fee = 0
        Status = "Unpaid"
    ]
}
```

C.14 Blueprint – Maintenance_Lifecycle_Blueprint

```
// DS-14 | Blueprint Maintenance_Lifecycle_Blueprint | After tab of
each transition
// Purpose: Set the Status field for the new stage (one line per
transition)
input.Status = "Assigned";      // Assign
input.Status = "In Progress";   // Start Work
input.Status = "Pending Approval"; // Send for Approval
input.Status = "Completed";     // Complete / Complete After Approval
input.Status = "Tenant Confirmed"; // Tenant Confirm
input.Status = "Closed";        // Close
```

C.2 [Form / Function Name]

```
// Script ID : [DS-02]
// Location  : [ ]
// Purpose   : [ ]

[paste code here]
```

APPENDIX D – USER MANUAL

D.1 Accessing the Application

Web: open the application link in Chrome or Edge and sign in with your Zoho account. Tenants: open the tenant portal address, enter the email address that received the invitation and the password you set. The mobile app was not tested.

D.2 Instructions for the Property Manager

- Open the Lease_Agreement form, choose the tenant and the unit, enter the dates and deposit, add co-occupants, have the tenant sign and submit. The unit becomes Occupied.
- Create a Rent_Invoice for the lease each month (or let the monthly schedule create it) and record each payment in Rent_Payment; the invoice is marked Paid and a receipt is emailed.
- Open Maintenance_Request records, use the blueprint buttons (Assign, Start Work and so on) to move each request forward, and add a Repair_Approval when a quote is above Rs 5,000.
- Decide approval items from the Approvals inbox, and review the RentEase_Dashboard for occupancy, rent, overdue invoices and open repairs.
- Use the Send Rent Reminders button on RentEase_Command_Center to email tenants with unpaid invoices.

D.3 Instructions for the Tenant

- Open the tenant portal address and sign in with the invited email and your password.
- Open Lease_Agreement, Rent_Invoice and Rent_Payment to view your lease, invoices and payments.
- Open Maintenance_Request and click the + button to raise a repair request with the category, description, priority and estimated cost.
- Open Move_Out_Request and click + to submit a move-out request when your lease ends.

APPENDIX E – EVALUATION RUBRIC

(For Examiner Use Only)

Student Name	Harini Sankar	Register No.	240701168
Project Title	RentEase: Property and Rent Management System	Date	10-11-2026

Table E.1: Evaluation Rubric for the Individual Project

S.No.	Criterion	Indicators	Max. Marks	Marks Awarded
1	Problem Definition & Requirement Analysis	Clarity of problem, measurable objectives, complete FR/NFR, use cases	10	
2	System Design	Architecture, ER/data model, form specifications, workflow design, permission matrix	10	
3	Forms, Reports & Pages	Variety of field types, lookups/sub forms, report types, UI Builder dashboards	15	
4	Deluge Scripting & Workflows	Correctness and depth of scripts; form, schedule, approval, Blueprint, batch workflows	15	
5	Integration, RBAC & AI	Zoho/external integrations, connections, roles & permissions, Zia/analytics	10	
6	Testing, Deployment, Security & Governance	Test coverage, environments, optimisation, security controls, audit	10	
7	Live Demonstration	End-to-end working of the application across roles; feature checklist verified	15	
8	Project Report	Adherence to template and format, clarity, figures, references	5	
9	Viva-Voce	Understanding of the platform, design decisions and low-code concepts	10	
	TOTAL		**100**	

Table E.2: Performance Level Descriptors

Level	% of Max. Marks	Descriptor
Excellent	90–100	Feature implemented completely, correctly and beyond the basic requirement; student explains design choices confidently.
Good	70–89	Feature implemented correctly with minor gaps; explanation adequate.
Satisfactory	50–69	Feature partially implemented or works only for the main path; limited explanation.
Needs Improvement	< 50	Feature missing, non-functional or not understood by the student.

Signature of Internal Examiner

Name: _____

Signature of External Examiner

Name: _____

APPENDIX F – VIVA-VOCE RECORD SHEET

(For Examiner Use Only)

Student Name	Harini Sankar	Register No.	240701168
Department / Section	CSE	Date of Viva	10-11-2026
Application URL Verified	Yes / No	Demo Duration	_____ min

Table F.1: Viva-Voce Questions and Responses

Q.No.	Question / Topic Asked	Unit	Response (E / G / S / NI)	Remarks
1				
2				
3				
4				
5				
6				
7				
8				

E – Excellent G – Good S – Satisfactory NI – Needs Improvement

Table F.2: Marks Summary

Component	Max. Marks	Marks Awarded
Criteria 1–8 (Appendix E)	90	
Viva-Voce	10	
TOTAL	**100**	

Examiners' Remarks:

_____

_____

INTERNAL EXAMINER **EXTERNAL EXAMINER**

Name: _____ Name: _____